

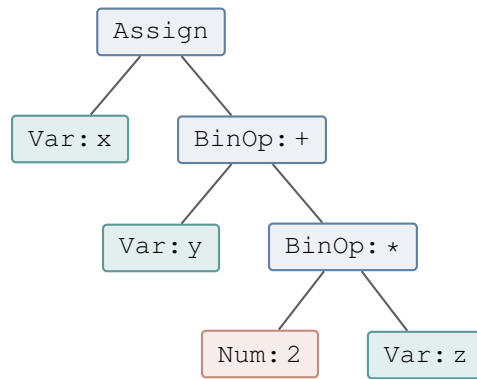
---

# Abstract Syntax Trees

## A Complete Theoretical Guide

---

From formal grammars to parsing algorithms,  
tree design, traversal, and semantic analysis



`x = y + 2 * z`

Prepared with Claude  
July 2026

*A theory-first treatment: what abstract syntax trees are, every major method  
for constructing them, and how they are used once built.*

# Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>What Is an Abstract Syntax Tree?</b>                                | <b>4</b>  |
| 1.1      | A first example . . . . .                                              | 4         |
| 1.2      | Where the AST lives: the language-processing pipeline . . . . .        | 5         |
| 1.3      | Why “abstract”? A preview of the central distinction . . . . .         | 6         |
| 1.4      | Who consumes ASTs . . . . .                                            | 6         |
| 1.5      | How this document is organized . . . . .                               | 7         |
| <b>2</b> | <b>Grammars: The Formal Theory of Syntax</b>                           | <b>8</b>  |
| 2.1      | Languages, strings, and alphabets . . . . .                            | 8         |
| 2.2      | The Chomsky hierarchy . . . . .                                        | 8         |
| 2.3      | Context-free grammars, formally . . . . .                              | 9         |
| 2.4      | Notation in practice: BNF and EBNF . . . . .                           | 10        |
| 2.5      | Ambiguity . . . . .                                                    | 10        |
| 2.6      | Encoding precedence and associativity in the grammar . . . . .         | 11        |
| 2.7      | A second classic: the dangling else . . . . .                          | 11        |
| 2.8      | Grammar hygiene: transformations that preserve the language . . . . .  | 12        |
| <b>3</b> | <b>From Parse Trees to Abstract Syntax</b>                             | <b>14</b> |
| 3.1      | The concrete syntax tree . . . . .                                     | 14        |
| 3.2      | The abstraction function . . . . .                                     | 15        |
| 3.3      | Abstract syntax as a first-class specification . . . . .               | 15        |
| 3.4      | One abstract syntax, many concrete syntaxes . . . . .                  | 16        |
| 3.5      | How abstract is abstract? A design spectrum . . . . .                  | 17        |
| <b>4</b> | <b>The Anatomy of an AST</b>                                           | <b>18</b> |
| 4.1      | The standard taxonomy of node kinds . . . . .                          | 18        |
| 4.2      | What every node should carry . . . . .                                 | 18        |
| 4.3      | Representation strategies . . . . .                                    | 19        |
| 4.4      | The expression problem . . . . .                                       | 21        |
| 4.5      | Immutability, sharing, and persistence . . . . .                       | 21        |
| <b>5</b> | <b>Lexical Analysis: From Characters to Tokens</b>                     | <b>23</b> |
| 5.1      | Tokens, lexemes, and token kinds . . . . .                             | 23        |
| 5.2      | Regular expressions and finite automata . . . . .                      | 23        |
| 5.3      | Disambiguation: maximal munch and priority . . . . .                   | 24        |
| 5.4      | The lexer–parser interface, and what it means for the AST . . . . .    | 24        |
| <b>6</b> | <b>Parsing I: The Landscape, and Top-Down Methods</b>                  | <b>26</b> |
| 6.1      | The landscape of parsing methods . . . . .                             | 26        |
| 6.2      | Top-down parsing: the idea . . . . .                                   | 26        |
| 6.3      | Recursive descent . . . . .                                            | 27        |
| 6.4      | The theory of prediction: NULLABLE, FIRST, FOLLOW, and LL(1) . . . . . | 28        |

|           |                                                                              |           |
|-----------|------------------------------------------------------------------------------|-----------|
| 6.5       | Backtracking, PEGs, and packrat parsing . . . . .                            | 29        |
| <b>7</b>  | <b>Parsing II: Operator-Precedence Methods</b>                               | <b>31</b> |
| 7.1       | The core question . . . . .                                                  | 31        |
| 7.2       | Dijkstra's shunting-yard algorithm . . . . .                                 | 31        |
| 7.3       | Precedence climbing . . . . .                                                | 32        |
| 7.4       | Pratt parsing (top-down operator precedence) . . . . .                       | 32        |
| 7.5       | Scope and limits . . . . .                                                   | 33        |
| <b>8</b>  | <b>Parsing III: Bottom-Up Methods — The LR Family</b>                        | <b>34</b> |
| 8.1       | Shift-reduce parsing and handles . . . . .                                   | 34        |
| 8.2       | LR(0): items and the characteristic automaton . . . . .                      | 34        |
| 8.3       | SLR(1), LR(1), LALR(1): the lookahead refinements . . . . .                  | 35        |
| 8.4       | Conflicts, and resolving them by declaration . . . . .                       | 36        |
| 8.5       | Building the AST: the value stack . . . . .                                  | 36        |
| 8.6       | GLR: bottom-up beyond determinism . . . . .                                  | 37        |
| <b>9</b>  | <b>Parsing IV: General Algorithms, Combinators, and Generators</b>           | <b>38</b> |
| 9.1       | Earley's algorithm . . . . .                                                 | 38        |
| 9.2       | CYK . . . . .                                                                | 38        |
| 9.3       | Parser combinators . . . . .                                                 | 39        |
| 9.4       | Parser generators and industrial tools . . . . .                             | 39        |
| 9.5       | Choosing a method . . . . .                                                  | 40        |
| <b>10</b> | <b>Building the Tree: Syntax-Directed Translation and Attribute Grammars</b> | <b>42</b> |
| 10.1      | Syntax-directed definitions . . . . .                                        | 42        |
| 10.2      | S-attributed SDDs and bottom-up parsing . . . . .                            | 42        |
| 10.3      | L-attributed SDDs and top-down parsing . . . . .                             | 43        |
| 10.4      | Attribute grammars in general . . . . .                                      | 43        |
| 10.5      | Translation schemes and marker nonterminals . . . . .                        | 44        |
| 10.6      | The payoff for AST builders . . . . .                                        | 44        |
| <b>11</b> | <b>Consuming the Tree: Traversal, Visitors, and Folds</b>                    | <b>45</b> |
| 11.1      | Traversal orders . . . . .                                                   | 45        |
| 11.2      | The visitor pattern . . . . .                                                | 45        |
| 11.3      | Folds: the mathematical shape of every pass . . . . .                        | 46        |
| <b>12</b> | <b>Semantic Analysis: Decorating the Tree</b>                                | <b>48</b> |
| 12.1      | Symbol tables and scopes . . . . .                                           | 48        |
| 12.2      | Name resolution . . . . .                                                    | 48        |
| 12.3      | Type checking . . . . .                                                      | 49        |
| 12.4      | The decorated tree . . . . .                                                 | 49        |

---

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>13 After the AST: Desugaring, Lowering, and Interpretation</b> | <b>50</b> |
| 13.1 Desugaring: AST-to-AST simplification . . . . .              | 50        |
| 13.2 Lowering: leaving the tree . . . . .                         | 50        |
| 13.3 Tree-walking interpretation . . . . .                        | 51        |
| <b>14 Advanced Topics: Errors, Fidelity, and Incrementality</b>   | <b>52</b> |
| 14.1 Syntax errors: detection, reporting, recovery . . . . .      | 52        |
| 14.2 Full-fidelity trees and the red–green architecture . . . . . | 53        |
| 14.3 Incremental parsing . . . . .                                | 53        |
| 14.4 The AST tooling ecosystem . . . . .                          | 53        |
| <b>15 A Builder’s Checklist</b>                                   | <b>55</b> |
| <b>Further Reading</b>                                            | <b>57</b> |

# 1 What Is an Abstract Syntax Tree?

Every program begins life as text: a flat, one-dimensional sequence of characters. Yet the *meaning* of a program is anything but flat. Expressions nest inside statements, statements nest inside functions, functions nest inside modules. The structure of a program is fundamentally *recursive* and *hierarchical*, and a linear string of characters is a poor vehicle for reasoning about it.

An **abstract syntax tree** (AST) is the data structure that resolves this mismatch. It is a tree-shaped representation of the syntactic structure of a program in which each node denotes a meaningful construct of the language—an addition, an assignment, a function definition, an `if` statement—and the children of a node denote that construct’s sub-parts. The tree is called *abstract* because it deliberately omits details of the concrete written form that carry no independent meaning: parentheses used only for grouping, semicolons, commas, keywords whose entire job is to delimit structure, whitespace, and comments. What remains is the essential grammatical skeleton of the program.

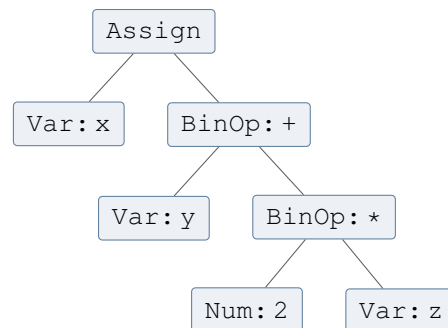
**Definition 1.1** (Abstract syntax tree). An abstract syntax tree is an ordered, rooted, labeled tree that represents the compositional structure of a program, in which nodes correspond to language constructs, children correspond to the constituents of those constructs, and purely notational elements of the source text are not represented.

## 1.1 A first example

Consider the assignment statement

$$x = y + 2 * z$$

As text, this is eleven characters (ignoring spaces). As structure, it is an assignment whose target is the variable `x` and whose value is the sum of `y` and the product of 2 and `z`. Notice that the structure already encodes a decision the flat text does not visibly make: multiplication binds more tightly than addition, so the right-hand side is  $y + (2 \cdot z)$ , not  $(y + 2) \cdot z$ . The AST makes this decision explicit and permanent:



Three properties of this picture deserve attention, because they are the essence of what makes ASTs useful.

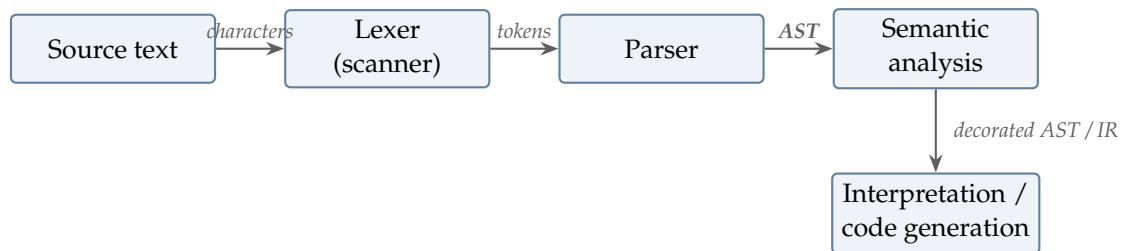
1. **Operator precedence has been compiled into shape.** In the source text, the fact that `*` outranks `+` is a convention the reader must know. In the tree, the `*` node simply *is* a child of the `+` node. Any tool that walks the tree gets precedence for free; no downstream consumer ever needs to think about it again.
2. **Notation has disappeared.** If the programmer had written `x = y + (2 * z)`, the tree would be *identical*. The parentheses changed nothing about the meaning, so they leave no trace. The AST is a quotient of the source text by “notational irrelevance.”
3. **The representation is compositional.** The meaning of the whole is a function of the meanings of the parts, and the tree shape mirrors exactly how the parts combine. This is what makes recursion the natural mode of computation over ASTs: to evaluate `Assign`, evaluate its value child; to evaluate `BinOp`, evaluate both children and combine.

#### Key idea

An AST is the point where a program stops being *text* and starts being *data*. Everything a compiler, interpreter, linter, formatter, or refactoring tool does is a computation over this data structure.

## 1.2 Where the AST lives: the language-processing pipeline

The AST is not produced in one leap from raw characters. It sits at a specific station in a pipeline whose stages have been essentially stable since the 1960s. Understanding this pipeline is a prerequisite for everything in this document, because each stage is governed by its own layer of theory.



- The **lexer** (Chapter 5) groups characters into **tokens**—the “words” of the language: identifiers, numbers, operators, keywords. Its theory is that of *regular languages* and finite automata.
- The **parser** (Chapters 6–9) consumes the token stream and discovers the hierarchical structure prescribed by the language’s *grammar*, producing the AST. Its theory is that of *context-free languages*, and it is the richest and most varied part of the story: there are many fundamentally different parsing algorithms, each with its own class of grammars it can handle, its own complexity guarantees, and its own way of constructing the tree.
- **Semantic analysis** (Chapter 12) walks the AST to resolve names, build symbol tables, and check types—attaching to the tree the information that syntax alone cannot express.

- Finally, the decorated tree is either *interpreted* directly (a “tree-walking interpreter”) or *lowered* into intermediate representations and ultimately machine code (Chapter 13).

The AST is thus the central artifact of the entire enterprise: the output of the front half of the pipeline and the input of the back half. Robert Nystrom, in *Crafting Interpreters*, describes the pipeline as a mountain: the front end *climbs* from low-level text up to the peak of abstraction—the AST—and the back end *descends* from that peak toward low-level executable form. Almost every question about language implementation is a question about how to get to the peak, what the peak should look like, or how to come down from it.

### 1.3 Why “abstract”? A preview of the central distinction

The word *abstract* is doing precise technical work, and it is defined by contrast with a second kind of tree. A grammar, applied mechanically, yields a **parse tree** (also called a **concrete syntax tree**, CST): a tree with one node for *every* grammar rule application and one leaf for *every* token, punctuation included. Parse trees are faithful but bloated; they record the derivation, not the meaning. The AST is what remains after we systematically discard everything in the parse tree that exists only to make the text parseable. Chapter 3 develops this distinction in full, because knowing *what to throw away* is the first genuine design decision in building an AST.

### 1.4 Who consumes ASTs

It is worth internalizing early how ubiquitous this data structure is, because it motivates the design pressures discussed later.

- **Compilers and interpreters** use the AST as the hub between parsing and evaluation or code generation. CPython, for instance, parses your source into an AST (inspectable via the standard `ast` module) before compiling it to bytecode.
- **Linters and static analyzers** (ESLint, PyLint, Clang-Tidy) pattern-match over AST nodes to detect suspicious constructs: an ESLint rule is essentially a set of callbacks fired when the traversal reaches nodes of particular types.
- **Transpilers and code transformers** (Babel, TypeScript’s compiler) parse source into an AST, *rewrite* the tree, and print it back out as code for a different target.
- **Formatters** (Prettier, Black, clang-format) parse code and re-print the tree according to layout rules—which is why they need trees that preserve, or can recover, comments.
- **Refactoring tools and codemods** perform large-scale automated edits by transforming trees rather than text, because tree transformations cannot produce syntactically invalid output.
- **IDEs and editors** maintain (often incrementally re-parsed) syntax trees to power highlighting, navigation, and completion; tree-sitter is the modern exemplar.

## 1.5 How this document is organized

The document is theory-first, as requested, and proceeds in the order in which the ideas logically depend on one another. Chapter 2 builds the formal foundation: grammars, derivations, ambiguity, precedence. Chapters 3 and 4 treat the tree itself—what abstraction means formally, and the full design space of node representations. Chapter 5 covers the lexical layer. Chapters 6 through 9 form the core survey of *every major method for building an AST*: recursive descent and the LL family; PEGs and packrat parsing; the operator-precedence family (shunting-yard, precedence climbing, Pratt); the LR family (LR(0), SLR, LALR, LR(1), GLR); the general algorithms (Earley, CYK); parser combinators; and parser generators. Chapter 10 explains the general theory of *syntax-directed translation* and attribute grammars—the formal account of how tree construction is woven into parsing. Chapters 11–13 cover what happens after the tree exists: traversal, the visitor pattern, semantic analysis, and lowering. Chapter 14 treats the modern frontier—error recovery, full-fidelity and incremental trees—and Chapter 15 distills everything into the concrete sequence of design decisions you face when specifying an AST of your own.



## 2 Grammars: The Formal Theory of Syntax

An AST cannot be defined, let alone built, without first answering the question: *what counts as a syntactically valid program?* The mathematical answer is a **formal grammar**. Grammars are the specification language for syntax; parsers are algorithms that decide membership in the language a grammar generates and recover the structure the grammar assigns. This chapter builds the theory from the ground up.

### 2.1 Languages, strings, and alphabets

**Definition 2.1** (Alphabet, string, language). An **alphabet**  $\Sigma$  is a finite, nonempty set of symbols. A **string** over  $\Sigma$  is a finite sequence of symbols from  $\Sigma$ ; the empty string is written  $\epsilon$ . The set of all strings over  $\Sigma$  is written  $\Sigma^*$ . A **language**  $L$  is any subset of  $\Sigma^*$ .

For a programming language, the interesting alphabet at the parsing level is not characters but the set of *token kinds* produced by the lexer (IDENT, NUM, PLUS, LPAREN, ...). “The set of all syntactically valid programs” is then a language over that alphabet—an infinite set, since programs can be arbitrarily long. Because the set is infinite, we cannot specify it by listing; we need a *finite generative device*. That device is the grammar.

### 2.2 The Chomsky hierarchy

Noam Chomsky’s 1956 classification arranges grammars into four nested classes by the form of their rewrite rules. Each class corresponds to a class of languages and to a class of abstract machines that recognize them.

| Type | Grammar class     | Rule form                                        | Recognizer               | Role in a compiler       |
|------|-------------------|--------------------------------------------------|--------------------------|--------------------------|
| 3    | Regular           | $A \rightarrow aB$ or $A \rightarrow a$          | Finite automaton         | Lexing (tokens)          |
| 2    | Context-free      | $A \rightarrow \alpha$                           | Pushdown automaton       | <b>Parsing (ASTs)</b>    |
| 1    | Context-sensitive | $\alpha A \beta \rightarrow \alpha \gamma \beta$ | Linear-bounded automaton | (rarely used directly)   |
| 0    | Unrestricted      | $\alpha \rightarrow \beta$                       | Turing machine           | (undecidable in general) |

The pivotal fact of language implementation is that **Type 2—the context-free grammars (CFGs)—is the sweet spot for syntax**. Regular grammars (Type 3) cannot express nesting: it is a classical theorem (provable with the pumping lemma) that the language of balanced parentheses is not regular, so no regular device can parse arithmetic expressions or block structure. Context-sensitive grammars are expressive enough but computationally expensive and hard to reason about. CFGs can express arbitrary nesting, are recognizable in at worst cubic time and in linear time for the subclasses used in practice, and—crucially for us—*every derivation in a CFG naturally has the shape of a tree*. The parse tree, and by extension the AST, is not an implementation convenience; it falls directly out of the mathematics of context-free derivation.

Real languages are not perfectly context-free. Rules like “variables must be declared before use” or “the number of arguments must match the declaration” are context-sensitive. The universal engineering resolution is to specify a slightly *over-generous* context-free syntax, build the AST

from it, and enforce the context-sensitive conditions afterward in semantic analysis (Chapter 12). Keep this division of labor in mind: *the grammar defines tree shapes; later passes define which trees make sense.*

## 2.3 Context-free grammars, formally

**Definition 2.2** (Context-free grammar). A context-free grammar is a quadruple  $G = (N, \Sigma, P, S)$  where  $N$  is a finite set of **nonterminal** symbols,  $\Sigma$  is a finite set of **terminal** symbols (tokens) with  $N \cap \Sigma = \emptyset$ ,  $P \subseteq N \times (N \cup \Sigma)^*$  is a finite set of **productions**, written  $A \rightarrow \alpha$ , and  $S \in N$  is the **start symbol**.

Nonterminals name syntactic categories (*expression, statement*); terminals are the tokens themselves; a production  $A \rightarrow \alpha$  says “one way an  $A$  can be built is as the sequence  $\alpha$ .” Several productions with the same left-hand side are conventionally written with the alternation bar:  $A \rightarrow \alpha \mid \beta$ .

**Example 2.1** (A grammar for arithmetic). Let  $\Sigma = \{\text{num}, \text{id}, +, *, (, )\}$  and consider:

|                       |        |                             |
|-----------------------|--------|-----------------------------|
| $E \rightarrow E + T$ | $\mid$ | $T$                         |
| $T \rightarrow T * F$ | $\mid$ | $F$                         |
| $F \rightarrow ( E )$ | $\mid$ | $\text{num} \mid \text{id}$ |

This little grammar, which will follow us through the entire document, generates exactly the well-formed arithmetic expressions over numbers and identifiers, and—as we will see in §2.6—its very shape encodes precedence and associativity.

### 2.3.1 Derivations

A grammar generates strings by **rewriting**. Starting from  $S$ , at each step we replace one nonterminal by the right-hand side of one of its productions. If  $\alpha A \gamma$  is a string of grammar symbols and  $A \rightarrow \beta$  is a production, we write  $\alpha A \gamma \Rightarrow \alpha \beta \gamma$  (“derives in one step”), and  $\Rightarrow^*$  for the reflexive–transitive closure. The **language of**  $G$  is  $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$ . A string of terminals and nonterminals reachable from  $S$  is a **sentential form**.

Two canonical disciplines fix which nonterminal to expand:

- a **leftmost derivation** always rewrites the leftmost nonterminal ( $\Rightarrow_{lm}$ );
- a **rightmost derivation** always rewrites the rightmost nonterminal ( $\Rightarrow_{rm}$ ).

This apparently bureaucratic distinction turns out to be the exact dividing line between the two great parsing families: *top-down parsers* (Chapter 6) *discover a leftmost derivation*; *bottom-up parsers* (Chapter 8) *discover a rightmost derivation in reverse*.

**Example 2.2.** A leftmost derivation of  $\text{id} + \text{num} * \text{id}$  in the grammar of Example 2.1:

$$E \Rightarrow_{lm} E + T \Rightarrow_{lm} T + T \Rightarrow_{lm} F + T \Rightarrow_{lm} \text{id} + T \Rightarrow_{lm} \text{id} + T * F \Rightarrow_{lm} \cdots \Rightarrow_{lm} \text{id} + \text{num} * \text{id}$$

### 2.3.2 Parse trees

A derivation is a *sequence*; the structure it reveals is better captured as a tree.

**Definition 2.3** (Parse tree). A **parse tree** (derivation tree) for  $G$  is a rooted ordered tree in which the root is labeled  $S$ , every interior node is labeled by a nonterminal  $A$  such that its children, read left to right, spell out the right-hand side of some production  $A \rightarrow \alpha$ , and every leaf is labeled by a terminal or  $\epsilon$ . The **yield** of the tree is the left-to-right sequence of its leaf labels.

A fundamental theorem connects the two views:  $w \in L(G)$  iff there exists a parse tree with yield  $w$ ; moreover, each parse tree corresponds to exactly one leftmost and exactly one rightmost derivation. The parse tree is the *derivation modulo the order of independent rewrites*—it captures precisely the structural content and forgets the bookkeeping. The AST will take one further quotient, forgetting the notational content too.

## 2.4 Notation in practice: BNF and EBNF

Grammars are written in **Backus–Naur Form** (BNF), introduced for the Algol 60 report, or its extended variant **EBNF**, which adds regular-expression-like operators purely as shorthand:

| EBNF notation                | Meaning         | Plain-BNF encoding                     |
|------------------------------|-----------------|----------------------------------------|
| $[ \alpha ]$ or $\alpha?$    | optional        | $X \rightarrow \alpha \mid \epsilon$   |
| $\{ \alpha \}$ or $\alpha^*$ | zero or more    | $X \rightarrow \alpha X \mid \epsilon$ |
| $\alpha^+$                   | one or more     | $X \rightarrow \alpha X \mid \alpha$   |
| $(\alpha \mid \beta)$        | grouping/choice | fresh nonterminal                      |

EBNF adds no expressive power—every EBNF grammar mechanically desugars to a CFG—but it maps pleasantly onto loops in hand-written parsers: a  $\{ \alpha \}$  repetition becomes a `while` loop rather than a recursive call, which (as §6.3.1 will show) is exactly how top-down parsers dodge the left-recursion problem.

## 2.5 Ambiguity

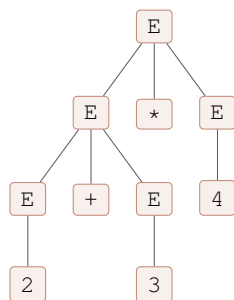
**Definition 2.4** (Ambiguity). A grammar  $G$  is **ambiguous** if some string in  $L(G)$  has more than one parse tree (equivalently, more than one leftmost derivation). A context-free *language* is **inherently ambiguous** if every grammar for it is ambiguous.

Ambiguity is the central disease of grammar design, because for us the parse tree is not a by-product—it is the *point*. Two trees mean two structures mean two meanings.

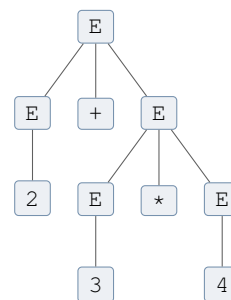
**Example 2.3** (The ambiguous expression grammar). The “naive” grammar

```
E → E + E | E * E | ( E ) | num
```

generates  $2 + 3 * 4$  with two distinct parse trees, corresponding to  $(2 + 3) \cdot 4 = 20$  and  $2 + (3 \cdot 4) = 14$ :



wrong:  $(2 + 3) * 4$



intended:  $2 + (3 * 4)$

Three sobering theoretical facts govern ambiguity. First, *ambiguity of a CFG is undecidable*: no algorithm can decide, for arbitrary  $G$ , whether  $G$  is ambiguous (a consequence of the undecidability of Post's Correspondence Problem). Practical tools therefore detect only sufficient conditions (e.g., LR conflict reports). Second, some context-free languages are *inherently* ambiguous, though real programming languages are designed to avoid this. Third, ambiguity is a property of the *grammar*, not the language: the language of Example 2.3 also has unambiguous grammars, and finding one is precisely the craft of the next section.

## 2.6 Encoding precedence and associativity in the grammar

The classical technique for disambiguating expression grammars is **stratification**: introduce one nonterminal per precedence level, with lower-precedence levels defined in terms of higher-precedence ones. The grammar of Example 2.1 is exactly this construction:

- $E$  (*expressions*) handles the loosest operator,  $+$ ;
- $T$  (*terms*) handles the tighter  $*$ ;
- $F$  (*factors*) handles atoms and parenthesized subexpressions.

Because a  $+$  can only appear via an  $E$ -production and its operands are an  $E$  and a  $T$ , a  $*$  can never end up *above* a  $+$  in the tree unless parentheses force it to. Precedence has been converted into *reachability structure* among nonterminals.

Associativity is encoded by *which side recurses*. In  $E \rightarrow E + T$ , the left operand is the recursive one, so  $a+b+c$  must parse as  $(a+b)+c$ : the grammar is **left-recursive** and yields *left associativity*. Dually, a right-recursive rule like  $R \rightarrow T ** R \mid T$  makes exponentiation right-associative, matching mathematical convention. This is elegant and important—and it plants a time bomb: Chapter 6 will show that top-down parsers *cannot handle left recursion at all*, forcing a delicate transformation that threatens to flip associativity unless tree construction is handled carefully. The tension between “the grammar that best expresses the language” and “the grammar a given algorithm can consume” is one of the running themes of parsing theory.

## 2.7 A second classic: the dangling else

Ambiguity is not confined to arithmetic. The grammar fragment

$S \rightarrow \text{if } E \text{ then } S$

```
S → if E then S else S
S → other
```

makes `if a then if b then s1 else s2` ambiguous: the `else` can attach to either `if`. Virtually all languages adopt the convention that an `else` binds to the *nearest* unmatched `then`. The convention can be enforced in the grammar itself by stratifying statements into *matched* (every `then` has its `else`) and *unmatched* forms:

```
S → M | U
M → if E then M else M | other
U → if E then S | if E then M else U
```

Alternatively—and far more commonly in practice—one keeps the ambiguous grammar and resolves the conflict *inside the parser*: LR parser generators, for instance, default to “shift on `else`,” which implements nearest-match directly (§8.4). This is a preview of an important practical dichotomy: disambiguation can live in the grammar (pure, verbose) or in the parsing algorithm’s tie-breaking rules (pragmatic, concise).

## 2.8 Grammar hygiene: transformations that preserve the language

Several mechanical transformations recur throughout parsing theory. Each preserves  $L(G)$  while changing the grammar’s *form* to suit an algorithm; each also perturbs the parse trees, which is why an AST-building parser must compensate (Chapter 10).

**Elimination of immediate left recursion.** Replace  $A \rightarrow A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n$  (where no  $\beta_i$  begins with  $A$ ) by

$$A \rightarrow \beta_1 A' \mid \cdots \mid \beta_n A' \quad A' \rightarrow \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon.$$

Intuitively: an  $A$  is “one  $\beta$  followed by a tail of  $\alpha$ ’s.” A generalization handles indirect left recursion ( $A \Rightarrow^* A\gamma$  through intermediate nonterminals) by ordering the nonterminals and substituting. Required by all top-down methods.

**Left factoring.** When two alternatives share a prefix,  $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ , factor it:  $A \rightarrow \alpha A'$ ,  $A' \rightarrow \beta_1 \mid \beta_2$ . This defers the choice until after the common prefix has been read—essential for parsers with one token of lookahead.

**$\varepsilon$ -removal, unit-production removal, and Chomsky Normal Form.** Any CFG (not generating  $\varepsilon$ ) can be converted to **CNF**, in which every production is  $A \rightarrow BC$  or  $A \rightarrow a$ . CNF is never used for production compilers, but it is the required input format of the CYK algorithm (§9.2) and the standard setting for proofs about CFGs.

### Key idea

A grammar is not merely a validity test; it is a *blueprint for tree shapes*. Precedence, associativity, and statement-nesting policy can all be compiled into the grammar’s structure—and every parsing algorithm in the chapters ahead is, at bottom, a strategy for reconstructing which

blueprint instantiation produced a given string.

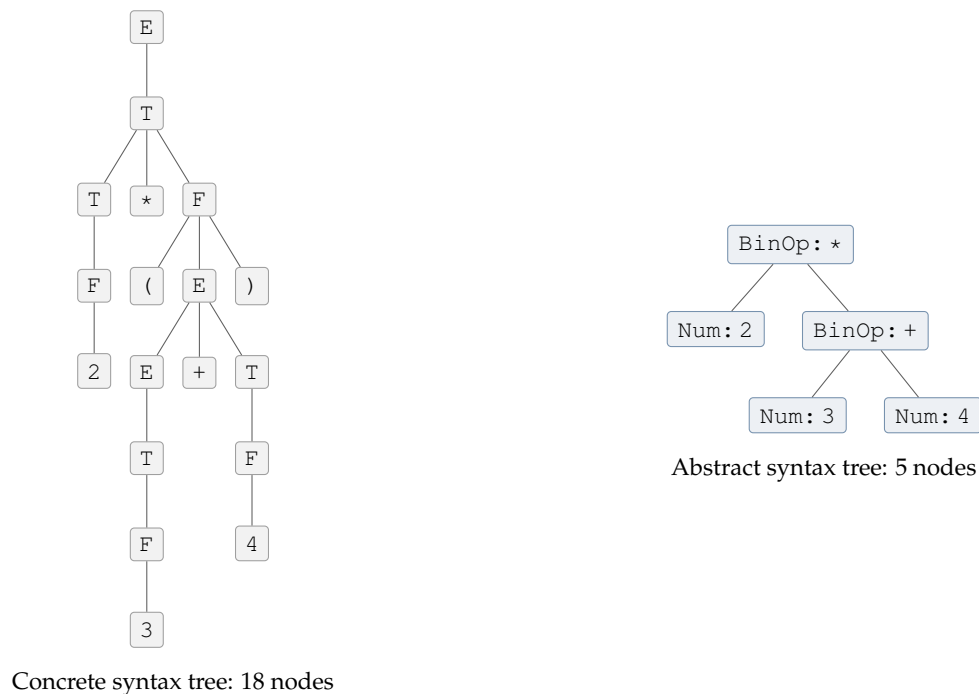
## 3 From Parse Trees to Abstract Syntax

Chapter 2 gave us the parse tree as the canonical structural witness that a string belongs to a language. This chapter performs the step that gives the AST its name: *abstraction*. We examine exactly what a parse tree records that an AST discards, why the discarding is principled rather than ad hoc, and how abstract syntax can be specified as a formal object in its own right—independent of any concrete grammar.

### 3.1 The concrete syntax tree

**Definition 3.1** (Concrete syntax tree). The **concrete syntax tree** (CST), or parse tree, of a string  $w$  with respect to grammar  $G$  is the derivation tree defined in Chapter 2: one interior node per production application, one leaf per token of  $w$  (including all punctuation), with the yield equal to  $w$  exactly.

The CST is *complete*: from it, the original token sequence—and with stored trivia, the original text—can be reconstructed character for character. But completeness has a price, visible the moment we draw one. Here is the CST of the humble expression  $2 * (3 + 4)$  under the stratified grammar of Example 2.1, beside its AST:



Every pathology of CSTs is on display in the left tree.

- **Chain nodes.** The path  $E \rightarrow T \rightarrow F \rightarrow 2$  exists only because the grammar routes every atom through all precedence levels. These unary chains say nothing about *this* program; they are fossils of the disambiguation technique of §2.6.

- **Punctuation leaves.** The tokens ( and ) appear as leaves. But their entire purpose—overriding precedence—has already been *achieved* by the shape of the tree. Once the + sits below the \*, the parentheses are informationally void.
- **Grammar-dependence.** The CST’s shape depends on which of the infinitely many equivalent grammars we chose. Refactor the grammar (left-factor it, eliminate left recursion for an LL parser) and every CST changes shape, even though the language and the meanings are untouched. Any tool written against CST shapes is coupled to a grammar’s incidental engineering.

## 3.2 The abstraction function

We can now say precisely what an AST is: the image of the CST under a systematic forgetting operation.

**Definition 3.2** (Abstraction, informally). An **abstraction function** maps each concrete syntax tree to an abstract one by (i) deleting leaves whose tokens are purely notational (delimiters, separators, keywords that only mark structure), (ii) collapsing chain nodes arising from precedence stratification or unit productions, (iii) replacing production-labeled interior nodes with construct-labeled nodes (an application of  $E \rightarrow E + T$  becomes a `BinOp` node carrying the operator + as data), and (iv) preserving exactly the operands, operators, names, and literals that determine meaning.

Two properties of this map deserve emphasis because they characterize the AST mathematically:

**It is many-to-one.** Distinct concrete strings— $2 * (3+4)$ ,  $(2 * (3+4))$ ,  $2 * (3 + 4)$ —map to the *same* AST. The AST is a canonical form: it identifies programs up to notational variation. This is precisely what downstream tools want, and precisely what a formatter must be careful about (Chapter 14), since a formatter’s job is to invert a non-injective function gracefully.

**It is meaning-preserving.** Nothing needed by evaluation, type checking, or translation is lost. A useful discipline when designing your own abstraction: *a token may be dropped iff the evaluator would never need to inspect it*. Parentheses pass the test; the operator symbol in `BinOp` obviously does not; a keyword like `while` passes because it is subsumed by the node’s own label.

### Key idea

CST : AST :: derivation : meaning-relevant structure. The CST answers “*how* was this string generated by *this* grammar?”; the AST answers “*what* was said?” Parsing is the computation of the first; abstraction is the quotient to the second; in practice the two are fused, and the parser emits ASTs directly (Chapter 10).

## 3.3 Abstract syntax as a first-class specification

Here is the conceptual move that elevates ASTs from “pruned parse trees” to an independent theoretical object: *abstract syntax has its own grammar*, one that never mentions tokens, precedence,



or punctuation at all. It simply declares, for each syntactic category, the constructs in that category and their components.

For our expression language:

```
expr = Num(int value)
      | Var(string name)
      | BinOp(expr left, oper op, expr right)
oper = Add | Mul
```

Read: “an expression is a number carrying an integer, or a variable carrying a name, or a binary operation carrying a left expression, an operator, and a right expression.” Three observations:

1. **This is an inductive (recursive) definition of a set of trees.** Formally, the set *Expr* is the least set closed under the constructors:  $\text{Num}(n) \in \text{Expr}$  for  $n \in \mathbb{Z}$ ;  $\text{Var}(s) \in \text{Expr}$  for strings  $s$ ; and if  $e_1, e_2 \in \text{Expr}$  and  $\odot \in \{\text{Add}, \text{Mul}\}$  then  $\text{BinOp}(e_1, \odot, e_2) \in \text{Expr}$ . Least-fixed-point definitions of this kind automatically license two proof/computation principles that we will lean on constantly: **structural induction** (to prove properties of all ASTs) and **structural recursion** (to define functions over all ASTs—evaluators, type checkers, printers). Chapter 11 develops the computational side as *folds*.
2. **This is an algebraic data type:** a *sum* (the alternatives, separated by `|`) of *products* (each constructor’s field list). The correspondence is exact, which is why languages with native algebraic data types and pattern matching (the ML family, Haskell, Rust, Scala, Swift) are famously pleasant for compiler writing, and why Chapter 4 treats “sum-of-products” as the semantic ground truth that every implementation strategy is encoding.
3. **It is real, deployed technology, not just a pedagogical device.** The notation above is essentially **ASDL** (the Abstract Syntax Description Language of Wang et al.), and CPython’s entire AST is defined in one ASDL file, `Python.asdl`, from which the interpreter’s C structures and the `ast` module’s Python classes are mechanically generated. An excerpt of the real thing:

```
stmt = FunctionDef(identifier name, arguments args, stmt* body, ...)
      | Return(expr? value)
      | Assign(expr* targets, expr value, ...)
      | While(expr test, stmt* body, stmt* orelse)
expr = BinOp(expr left, operator op, expr right)
      | Call(expr func, expr* args, keyword* keywords)
      | Name(identifier id, expr_context ctx)
      | Constant(constant value, string? kind)
```

The `*` marks a sequence field and `?` an optional field—ASDL’s only two type constructors beyond products and sums, and empirically all you need.

### 3.4 One abstract syntax, many concrete syntaxes

Because the abstract grammar never mentions notation, it is *independent of surface syntax*. The same abstract `BinOp(Num 2, Mul, BinOp(Num 3, Add, Num 4))` is the meaning-structure of

the infix  $2 * (3 + 4)$ , the prefix (Lisp)  $( * \ 2 \ (+ \ 3 \ 4) )$ , and the postfix (RPN)  $2 \ 3 \ 4 \ + \ *$ . One can build several parsers—several *concrete* front ends—targeting a single abstract syntax, and everything downstream is shared. Conversely, a **pretty-printer** or **unparser** maps abstract syntax back to some concrete syntax, reinserting the minimal parentheses that precedence demands. The pair (parser, pretty-printer) exhibits the AST as the fixed hub between notation and meaning.

This is also the right lens on *source-to-source* tools: a transpiler such as Babel or the TypeScript compiler is  $\text{parse} \rightarrow \text{transform-the-tree} \rightarrow \text{unparse}$ , and a formatter is the degenerate transpiler whose tree transformation is the identity.

### 3.5 How abstract is abstract? A design spectrum

“Abstract” is not a single point but a dial, and where you set it is a genuine design decision governed by your consumers:

**Maximally abstract** (classic compiler textbooks): drop everything non-semantic, even desugar during parsing (e.g., translate `for` into `while` on the way in). Smallest trees, simplest back end; but source locations must be carried as explicit annotations, and faithful error messages or refactoring become impossible.

**Conventional AST** (CPython, most compilers): drop punctuation and chains, keep every distinct source construct as its own node kind, annotate every node with a source *span* (start/end line and column). Desugaring happens in a later, visible pass. This is the default recommendation.

**Full-fidelity / lossless tree** (Roslyn, LibCST, tree-sitter): keep *everything*, including whitespace and comments (“trivia”), so that the original text is reproducible byte-for-byte, while still exposing an abstract-shaped API. Mandatory for formatters, IDE refactorings, and codemods; costlier and more complex. Chapter 14 examines the red–green tree architecture that makes this efficient.

The rule of thumb: *compilers want the middle setting; tools that write code back out want the third; only throwaway interpreters want the first*. Whatever the setting, the theory is unchanged—each is an algebraic data type of trees, built by the methods of Chapters 6–9 and consumed by the recursion schemes of Chapter 11.

## 4 The Anatomy of an AST

We now know *what* an AST is mathematically: an element of an inductively defined sum-of-products type. This chapter surveys the design space for realizing that mathematics as a concrete data structure—the taxonomy of node kinds a real language needs, the information each node should carry, the major representation strategies with their trade-offs, and the deep design tension known as the *expression problem*. These decisions matter even if “AI will build it”: they determine what every later pass can and cannot do cheaply.

### 4.1 The standard taxonomy of node kinds

Real languages organize their abstract syntax into a small number of top-level **syntactic categories**, each an algebraic sum of its own. Almost every language’s AST is a variation on this scheme:

**Expressions** — constructs that *produce a value*: literals (`Num`, `Str`, `Bool`), variable references (`Var/Name`), unary and binary operations, function calls (`Call`), indexing and field access, conditionals-as-expressions, lambdas. Expressions are recursive in themselves: the children of an expression node are expressions.

**Statements** — constructs that *perform an effect* and produce no value: assignment, `if/while/for`, `return`, `break`, blocks. Statements typically contain both expressions (a `while`’s test) and other statements (its body), giving the two categories a mutually recursive structure.

**Declarations / definitions** — constructs that *introduce names*: function, variable, class, and module definitions. Some languages fold these into statements; others keep them distinct because they alone affect scope.

**Types** — when the surface language has type annotations, these get their own category of nodes (`TypeName`, `FunctionType`, `GenericInstantiation`, ...).

**Patterns** — in languages with destructuring or `match`, the left-hand sides form yet another category, structurally similar to expressions but semantically dual (they *consume* values and *bind* names).

**Auxiliary structures** — parameter lists, argument lists, import clauses: product-only helper types that appear as fields of the above.

The categories matter theoretically because they are *separate inductive types*: an evaluator for expressions returns values, an executor for statements returns (say) an environment or a control signal, and the type system of your implementation language can enforce that a statement is never placed where an expression belongs—one of the strongest correctness levers available to a compiler writer. Expression-oriented languages (Lisp, Rust, ML) deliberately collapse the expression/statement distinction, simplifying the AST at the cost of surface-language familiarity.

### 4.2 What every node should carry

Beyond its children, a well-designed node carries a small amount of non-structural payload.

**Source location (span).** At minimum a start and end position (offset, or line/column) covering the node’s extent in the original text. Locations are semantically inert but operationally indispensable: error messages, debugger stepping, coverage, and source maps all depend on them, and they cannot be recovered later if dropped at parse time. The universal design is a `Span` attached uniformly to every node, assembled by the parser as it goes (Chapter 10).

**Interned identifiers.** Names are best stored as **interned symbols** (indices into a global string table) rather than raw strings: equality becomes integer comparison, and memory is shared. Interning is a lexer-level concern with AST-level payoff.

**Slots for later decoration.** Semantic analysis (Chapter 12) will want to attach resolved symbols to `Var` nodes and inferred types to every expression. The design choice—mutable fields filled in later, a side table keyed by node identity, or a rebuilt *typed* tree—is taken up in §4.5 and Chapter 12.

**Node identity.** Whether two structurally equal subtrees are “the same node” matters for side tables and for sharing. Pointer-based trees get identity for free; value-based and flattened designs must decide explicitly.

#### Pitfall

The single most common beginner mistake in AST design is omitting source spans “to keep it simple,” then discovering that every diagnostic reads “*type error somewhere*.” Locations must flow from tokens into nodes from day one; retrofitting them touches every constructor call in the parser.

## 4.3 Representation strategies

The same abstract type—say `Expr` from §3.3—admits several fundamentally different machine representations. All are in production use; the differences are ergonomic and operational, not semantic.

### 4.3.1 Heterogeneous object trees (the OO encoding)

One class per constructor, all deriving from an abstract base per category:

```
abstract class Expr { Span span; }
class Num extends Expr { int value; }
class VarRef extends Expr { Symbol name; }
class BinOp extends Expr { Expr left; Op op; Expr right; }
```

Each node knows its precise field types (“heterogeneous”), and the host type system rejects malformed trees at compile time. Operations over the tree are written either as virtual methods on the classes (which scatters one conceptual pass across many files) or, far more commonly in compilers, via the **visitor pattern** (Chapter 11), which restores per-pass locality. This is the Java/C#/C++ mainstream, and the style *Crafting Interpreters* teaches—complete with a small code generator that writes the boilerplate class definitions, an early taste of “the AST definition is itself generated from a specification” (cf. ASDL).

### 4.3.2 Algebraic data types (the functional encoding)

A direct transcription of the abstract grammar:

```
data Expr = Num Int
          | VarRef Symbol
          | BinOp Expr Op Expr
```

Operations are single functions with **pattern matching** over constructors—the programming-language reification of structural recursion. Exhaustiveness checking (the compiler warns if a new constructor is unhandled) is a decisive safety feature during language evolution. Rust’s `enum + Box` sits here; so do OCaml and Haskell, the traditional compiler-writing languages.

### 4.3.3 Homogeneous trees (the dynamic encoding)

One generic node type—a tag plus a list of children—used for every construct:

```
class Node { string kind; list<Node> children; map attrs; }
```

This is the shape of Lisp *S-expressions* (where the program literally *is* its own AST—the property called **homoiconicity**, which makes Lisp macros a form of user-level AST transformation), of JSON ASTs such as ESLint/Babel’s ESTree format, and of parse trees produced by generic tools (ANTLR’s default trees, tree-sitter). Maximum flexibility and trivial serialization, zero static shape checking: nothing stops a `BinOp` with five children. Suitable for dynamically typed hosts and for interchange between tools.

### 4.3.4 Flattened / arena representations (the data-oriented encoding)

Modern high-performance compilers (Zig’s, Carbon’s, rustc internally) store all nodes of a tree contiguously in one array; “child pointers” become 32-bit indices into the array:

```
struct Node { NodeKind kind; Span span; u32 lhs; u32 rhs; }
tree: array<Node>           // e.g., in post-order
```

Benefits: dramatically better cache locality and memory density, trivial bulk (de)allocation, stable node IDs (the index) for side tables, and cheap serialization. Costs: less natural to construct and mutate, and the host type system no longer distinguishes expression indices from statement indices unless wrapped. A notable theoretical bonus: if nodes are stored in the order the parser produces them, a post-order array *is* a linearized bottom-up traversal, and some passes can run as a single linear scan without pointer chasing at all.

### 4.3.5 Comparison

|                     | Heterogeneous OO | Algebraic (ADT)      | Homogeneous      | Flattened/arena   |
|---------------------|------------------|----------------------|------------------|-------------------|
| Static shape safety | strong           | strongest            | none             | weak              |
| Adding a new pass   | visitor          | new function         | new function     | new scan          |
| Adding a node kind  | easy             | edit all matches     | trivial          | edit all switches |
| Memory locality     | poor             | poor–fair            | poor             | excellent         |
| Serialization       | manual           | derivable            | trivial          | trivial           |
| Typical hosts       | Java, C#, C++    | OCaml, Haskell, Rust | JS, Python, Lisp | Zig, Rust, C      |

## 4.4 The expression problem

The second and third rows of that table are two faces of a famous design tension, named the **expression problem** by Philip Wadler. An AST library has two axes of extension: adding new *node kinds* (data variants) and adding new *operations* (passes). The two mainstream encodings each make exactly one axis easy and the other invasive:

- In the **OO encoding**, adding a variant is modular (one new subclass, existing code untouched), but adding an operation means touching every class—unless you adopt the visitor pattern, which *flips* the axes: now operations are modular (one new visitor) and variants are invasive (every visitor interface grows a method).
- In the **ADT encoding**, adding an operation is modular (one new function), but adding a variant means revisiting every pattern match—which exhaustiveness checking at least turns into a compiler-guided todo list.

For compilers the resolution is empirical: *a language’s set of node kinds stabilizes early, while new passes are added forever*. Hence the ADT/visitor side of the trade is almost always right, which is exactly why the visitor pattern (Chapter 11) is the signature idiom of OO compiler code. Full solutions to the expression problem (object algebras, tagless-final encodings, open recursion) exist and are elegant, but production compilers rarely need the generality.

## 4.5 Immutability, sharing, and persistence

Should tree nodes be mutable? The theory-driven answer favors **immutability**: an immutable tree can share subtrees freely (two occurrences of the same subexpression can be one object), supports *persistent* update—rebuilding only the spine from a changed node to the root in  $O(\text{depth})$  while sharing everything else—and makes concurrent traversal trivially safe. These properties underlie the red–green architecture of Chapter 14 and the functional-transform style of tools like Babel plugins. The price is that “decorating” the tree during semantic analysis cannot be done in place; one instead (a) rebuilds a new, more informative tree (an untyped AST becomes a *typed* AST—in richly typed hosts these can even be one GADT-indexed family), or (b) keeps decorations in side tables keyed by node identity. Mutable single-owner trees, filled in by successive passes, remain a fine pragmatic choice for a single-threaded compiler; the point is to choose *deliberately*, because the choice silently determines your options for incrementality, parallelism, and tooling later.

**Key idea**

An AST design is four decisions: (1) the abstract grammar—which categories and constructors exist; (2) the per-node payload—spans always, identity policy, decoration policy; (3) the encoding—objects, ADTs, homogeneous, or flattened; (4) mutability. Everything in Chapters 11–14 is downstream of these four choices, and Chapter 15 turns them into a checklist.

## 5 Lexical Analysis: From Characters to Tokens

Parsers do not read characters; they read **tokens**. The lexer (scanner, tokenizer) is the pipeline stage that performs this pre-digestion, and it rests on a different, simpler layer of theory than parsing: the theory of *regular languages*. This chapter covers that theory, the disambiguation conventions every real lexer uses, and the design questions at the lexer–parser boundary that directly affect the AST.

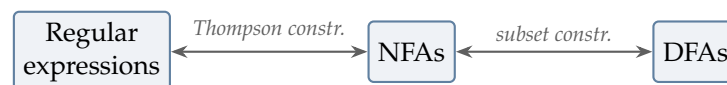
### 5.1 Tokens, lexemes, and token kinds

**Definition 5.1** (Token). A **token** is a pair (kind, value) produced by the lexer: the **kind** is a symbol from a finite set (`IDENT`, `NUM`, `PLUS`, `LPAREN`, `KW_IF`, ...), and the **value** (or attribute) carries the kind-specific payload—the identifier’s interned name, the numeric literal’s value. The exact substring of source text matched is the **lexeme**. Every token also records its source span.

The token kinds form the terminal alphabet  $\Sigma$  of the grammar in Chapter 2: the lexer is precisely the adapter between “language over characters” and “language over tokens.” Splitting the work this way is a separation-of-concerns theorem in action: token shapes are regular (no nesting), so they can be recognized by finite automata in linear time with no stack, while the parser reserves its stack-powered machinery for genuinely nested structure. The split also concentrates all character-level messiness—whitespace, comments, escape sequences, Unicode—in one place, so the grammar stays clean.

### 5.2 Regular expressions and finite automata

Each token kind is specified by a **regular expression**. The core theory is a triangle of equivalences established by Kleene, Rabin–Scott, and Thompson:



Every regular expression converts (Thompson’s construction) into a **nondeterministic finite automaton** (NFA) of linear size; every NFA converts (the subset construction) into a **deterministic finite automaton** (DFA), possibly with exponentially many states but recognizing input in strict  $O(n)$  time with  $O(1)$  memory; and DFAs can be minimized to a canonical smallest form. A production lexer is conceptually one big DFA: the union of all token regexes, with each accepting state labeled by the token kind it recognizes. Lexer generators (`lex/flex`, or the lexer modes of ANTLR) literally perform these constructions; hand-written lexers implement the same DFA as a loop with a switch on the current character—straightforward enough that many production compilers write theirs by hand for the sake of error messages and speed.

The theoretical boundary is as important as the machinery: by the **pumping lemma for regular languages**, finite automata cannot count unboundedly, so nested constructs (balanced parentheses, nested block comments) are provably beyond the lexer. Languages that allow nested



comments either handle them with a small ad hoc counter in the lexer (stepping quietly outside the regular fence) or forbid nesting.

### 5.3 Disambiguation: maximal munch and priority

The token regexes overlap—`if` matches both the keyword rule and the identifier rule; `<=` contains `<`. Two universal conventions resolve all such conflicts deterministically:

1. **Maximal munch (longest match).** At each step the lexer emits the *longest* prefix of the remaining input that matches any token rule. Hence `<=` lexes as one token, `forx` as one identifier rather than `for + x`. Implementation-wise this means running the DFA past the last accepting state until it dies, then backtracking to that last accepting position.
2. **Rule priority.** Among rules matching the same longest lexeme, the earliest-declared wins. Keywords are declared before the identifier rule (or, equivalently, identifiers are lexed first and then filtered through a keyword table).

Maximal munch is a convention, not a law of nature, and it occasionally bites language designers: in early C++, nested template types like `list<list<int>>>` could not be written without a space, because `>>` maximally munched as the shift operator—fixed only by special-casing the parser context. Such episodes are useful reminders that lexer-level decisions have grammar-level consequences.

### 5.4 The lexer–parser interface, and what it means for the AST

Several boundary-layer decisions shape the tree that ultimately gets built:

**Trivia policy.** Whitespace and comments are skipped by classical lexers—correct for a compiler, fatal for a formatter. Full-fidelity systems instead attach skipped **trivia** to adjacent tokens (Roslyn’s leading/trailing trivia model) so the eventual tree is lossless (§3.5, Chapter 14).

**Values and interning.** Converting lexemes to values (parsing digits into integers, decoding string escapes, interning identifiers) belongs in the lexer, so AST literal nodes carry ready-to-use payloads.

**Spans.** Tokens carry the positions from which node spans (§4.2) will be assembled; the lexer is the sole source of positional truth.

**Layout-sensitive syntax.** Python’s significant indentation and Haskell’s layout rule are handled by a lexer post-pass that compares indentation levels and synthesizes virtual `INDENT` and `DEDENT` (or brace) tokens. This keeps the *grammar* context-free—the parser just sees balanced virtual delimiters—at the price of a stateful lexer. It is the standard trick for reconciling human-friendly layout with CFG-friendly parsing.

**Scannerless parsing.** Some formalisms (notably PEGs, §6.5, and systems like SGLR) dispense with the separate lexer and parse characters directly. The gain is compositionality (embedding one language in another without token-set clashes); the cost is that the grammar must now legislate whitespace explicitly everywhere. The conventional two-level design remains the default for good reason.

**Key idea**

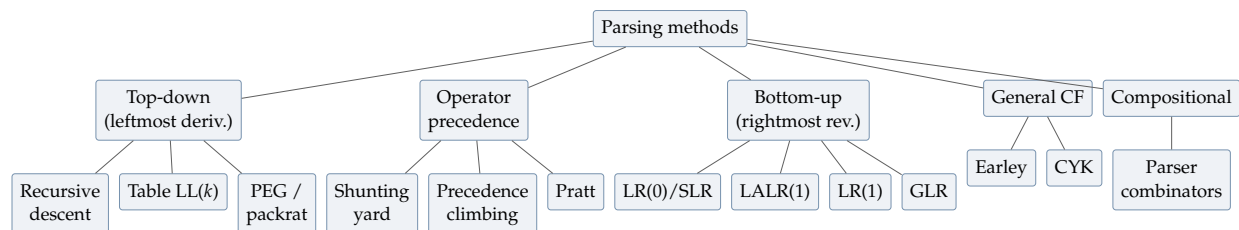
The lexer is a compiled disjunction of regular expressions—a DFA—running in linear time, whose output alphabet is the parser’s input alphabet. Its two quiet superpowers for AST construction are *spans* and *values*: every location and every literal in the final tree originates here.

## 6 Parsing I: The Landscape, and Top-Down Methods

We arrive at the heart of the matter: given a grammar and a token stream, *reconstruct the tree*. This chapter first maps the whole territory—every major family of parsing methods and how they relate—then develops the top-down family in depth: recursive descent, the  $LL(k)$  theory that underpins it, and the modern PEG/packrat alternative.

### 6.1 The landscape of parsing methods

All practical parsing algorithms answer the same question—*which derivation produced this string?*—but differ along four axes: the *direction* in which they build the tree (from the root down, or from the leaves up), the *grammar class* they accept, their *complexity*, and their *determinism* (single guaranteed parse vs. exploration of alternatives). The family tree:



A coarse but reliable orientation before the details:

- **Top-down** methods are the ones humans write by hand; they read like the grammar and give total control, but choke on left recursion and need grammars massaged into predictable form.
- **Operator-precedence** methods (Chapter 7) are specialized engines for the expression sub-language, driven by precedence tables instead of stratified grammars; almost every serious hand-written parser embeds one.
- **Bottom-up LR** methods (Chapter 8) accept the largest deterministic grammar class—left recursion included—and are the classical territory of parser *generators*.
- **General** methods (Chapter 9) accept *every* CFG, ambiguity and all, at higher asymptotic cost; they power natural-language work and grammar prototyping.
- **Combinators** are a library-design technique that can package any of the above, most naturally the top-down family.

### 6.2 Top-down parsing: the idea

A top-down parser reconstructs the derivation *in the order it was made*: it starts by assuming the whole input is an  $S$ , chooses a production for  $S$ , and recursively does the same for each

nonterminal in that production, consuming tokens left to right. It is therefore building a **leftmost derivation**—and, equivalently, constructing the parse tree from the *root downward*, pre-order. The whole game is the word “chooses”: when a nonterminal has several productions, the parser must *predict* the right one, ideally from the next  $k$  unread tokens. Parsers that can always do so with  $k$  tokens of lookahead are called **predictive**, and the grammars that permit it are the **LL( $k$ )** grammars—“scan Left-to-right, produce a Leftmost derivation, with  $k$  lookahead.”

## 6.3 Recursive descent

**Recursive descent** is the direct transcription of a grammar into mutually recursive procedures: one procedure per nonterminal, whose body mirrors the right-hand sides—a call for each nonterminal, a token-match for each terminal, an *if/switch* on lookahead for each alternation, a loop for each EBNF repetition. It is the oldest technique, the one used by most production compilers today (GCC, Clang, Go, Rust, TypeScript, V8 all hand-write their parsers), and the one *Crafting Interpreters* builds, precisely because the code *is* the grammar.

Using the (left-recursion-free, see below) expression grammar

|                             |                             |                                                      |
|-----------------------------|-----------------------------|------------------------------------------------------|
| $E \rightarrow T \{ + T \}$ | $T \rightarrow F \{ * F \}$ | $F \rightarrow ( E ) \mid \text{num} \mid \text{id}$ |
|-----------------------------|-----------------------------|------------------------------------------------------|

an AST-producing recursive-descent parser is:

```
function parseE() : Expr
  left := parseT()
  while lookahead = PLUS do
    consume(PLUS)
    right := parseT()
    left := BinOp(left, Add, right)    - grows tree leftward
  return left

function parseF() : Expr
  if lookahead = LPAREN then
    consume(LPAREN); e := parseE(); consume(RPAREN); return e
  elif lookahead = NUM then return Num(consumeValue())
  elif lookahead = ID then return VarRef(consumeValue())
  else raise SyntaxError(expected: "(", num, id)
```

Three theoretical points are folded into this small listing:

1. **The call stack is the parse stack.** The pushdown automaton that context-free recognition provably requires is implemented for free by the host language’s procedure stack; the chain of active calls at any moment *is* the path from the tree root to the current node.
2. **Grouping parentheses vanish by construction.** `parseF` returns the inner expression itself—no `Paren` node—executing the abstraction function of Chapter 3 on the fly. Building the AST directly, rather than a CST first, is the norm.
3. **The loop, not recursion, restores left associativity.** This deserves its own subsection.

### 6.3.1 Left recursion, and the associativity trap

The natural grammar rule  $E \rightarrow E + T \mid T$  is **left-recursive**, and a top-down parser for it—`parseE` calling `parseE` as its very first act, no token consumed—recurses forever. Top-down methods therefore demand the left-recursion elimination of Chapter 2:

$$E \rightarrow T E' \quad E' \rightarrow + T E' \mid \varepsilon$$

But this rewritten grammar is *right-recursive*, and its parse tree for `a+b+c` hangs to the right—naively folding it into `BinOp` nodes yields `a+(b+c)`, the wrong associativity for subtraction and division. The standard resolution, visible in `parseE` above, is to implement  $E'$ 's tail recursion as a *loop* that repeatedly wraps the accumulated tree as the new *left* operand: `left := BinOp(left, op, right)`. The grammar transformation changes derivation shape; the *semantic action* compensates to restore the intended tree. This is the single most instructive example of a theme formalized in Chapter 10: the parse structure and the constructed AST need not coincide, and the mapping between them is programmable.

## 6.4 The theory of prediction: NULLABLE, FIRST, FOLLOW, and LL(1)

When is naive prediction—one token of lookahead, no backtracking—guaranteed to work? The answer is the LL(1) condition, built from three computable sets. Fix a grammar  $G$ ; let  $\alpha, \beta$  range over strings of grammar symbols and  $A$  over nonterminals.

**Definition 6.1.**

$$\begin{aligned} \text{NULLABLE}(\alpha) &\iff \alpha \Rightarrow^* \varepsilon \\ \text{FIRST}(\alpha) &= \{ a \in \Sigma \mid \alpha \Rightarrow^* a\gamma \text{ for some } \gamma \} \\ \text{FOLLOW}(A) &= \{ a \in \Sigma \mid S\$ \Rightarrow^* \alpha A a \beta \text{ for some } \alpha, \beta \} \end{aligned}$$

where  $\$$  is the end-of-input marker (so  $\$ \in \text{FOLLOW}(S)$ ).

$\text{FIRST}(\alpha)$  collects the tokens that can *begin* something derived from  $\alpha$ ;  $\text{FOLLOW}(A)$  collects the tokens that can legally *come after* an  $A$ —which is exactly what a parser must consult to decide whether to apply an  $\varepsilon$ -production (finish this  $A$  now) or keep going. All three are computed by a simple least-fixed-point iteration over the productions: initialize everything empty, repeatedly apply the closure rules (e.g., for  $A \rightarrow Y_1 \cdots Y_k$ , add  $\text{FIRST}(Y_1)$  to  $\text{FIRST}(A)$ ; if  $Y_1$  is nullable, also add  $\text{FIRST}(Y_2)$ ; if all  $Y_i$  nullable, mark  $A$  nullable; symmetric rules propagate  $\text{FOLLOW}$  rightward and from  $A$  to a trailing nullable suffix), until nothing changes. Termination is guaranteed because the sets only grow and are bounded.

**Definition 6.2** (LL(1) grammar).  $G$  is **LL(1)** iff for every pair of distinct productions  $A \rightarrow \alpha$  and  $A \rightarrow \beta$ :

1.  $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ ;
2. at most one of  $\alpha, \beta$  is nullable; and
3. if  $\beta \Rightarrow^* \varepsilon$ , then  $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ .

The three clauses say precisely: *one token of lookahead always selects a unique production*. From the sets one mechanically fills a **predictive parsing table**  $M[A, a] = \text{“the production to apply when expanding } A \text{ with lookahead } a\text{”}$ : for each  $A \rightarrow \alpha$ , set  $M[A, a] = A \rightarrow \alpha$  for every  $a \in \text{FIRST}(\alpha)$ , and if  $\alpha$  is nullable, also for every  $a \in \text{FOLLOW}(A)$ . The grammar is LL(1) iff no table cell receives two entries. The table can then drive either the `switch` statements of a recursive-descent parser or an explicit-stack, non-recursive **table-driven LL parser** (push  $S$ ; repeatedly pop, matching terminals against input and replacing nonterminals via  $M$ )—the two are the same automaton in different clothing.

Immediate corollaries connect back to grammar hygiene: a left-recursive grammar is never LL(1) (clause 1 fails— $\text{FIRST}(A\alpha) \supseteq \text{FIRST}(\beta)$ ), and an unfactored common prefix is never LL(1); hence the two transformations of Chapter 2 are not stylistic advice but exactly the repairs the LL(1) definition demands. **LL( $k$ )** generalizes all of this to  $k$ -token lookahead sets; every LL( $k$ ) grammar is unambiguous, and the LL( $k$ ) classes form a strict hierarchy in  $k$ . Modern top-down generators such as ANTLR implement **LL( $*$ )**—adaptive, effectively unbounded regular lookahead—so that grammar authors rarely think about  $k$  at all; the classical theory above is what such tools automate.

## 6.5 Backtracking, PEGs, and packrat parsing

What if prediction fails—if no finite lookahead distinguishes two alternatives? The old answer is **backtracking**: try the first alternative; on failure, rewind the input and try the next. Naive backtracking is exponential in the worst case and, worse, makes error reporting miserable (the “failure” that matters is buried under speculative ones). The modern rehabilitation of backtracking is Bryan Ford’s **parsing expression grammars** (PEGs, 2004), which change the *semantics* of a grammar rather than the algorithm:

- Choice is **ordered**:  $e_1 / e_2$  means “try  $e_1$ ; only if it fails, try  $e_2$ .” A PEG is therefore *never ambiguous by definition*—the first matching alternative wins—at the price that the grammar author must reason about ordering (the classic trap: “<” / “<=” never matches “=”).
- Repetition ( $e^*$ ) is *greedy and possessive*: it consumes as much as possible and never gives any back.
- Syntactic **predicates**  $\&e$  (and-predicate: succeed iff  $e$  would match, consuming nothing) and  $!e$  (not-predicate) grant unbounded lookahead, taking PEGs beyond context-free recognition power in some respects (e.g.,  $a^n b^n c^n$  is PEG-recognizable) while, remarkably, it remains an open problem whether every context-free language is PEG-recognizable.
- Pure PEGs cannot express left recursion (same infinite-descent problem), though published extensions and several implementations support it via bounded growing of a memoized seed.

Ford’s companion result makes PEGs practical: **packrat parsing** memoizes, for every (grammar-expression, input-position) pair, the result of the attempt—success with extent and value, or failure. Since there are  $O(|G| \cdot n)$  such pairs and each is computed once, *any* PEG parses in linear time, backtracking notwithstanding, at the cost of memory proportional to input size (the practical constant is large, and many real PEG implementations memoize selectively). PEGs also compose without a separate lexer (scannerless, cf. Chapter 5). Their significance for us: CPython

replaced its aging LL(1) parser with a PEG parser in Python 3.9 precisely to escape LL(1)'s grammar contortions—an instructive case study in the trade between grammar-class elegance and grammar-authoring freedom.

#### Key idea

Top-down parsing is *prediction*: choose a production now, justify it with lookahead. Its theory (NULLABLE/FIRST/FOLLOW and the LL hierarchy) tells you exactly when one-token prediction suffices; its craft (loops for left associativity, embedded precedence engines—next chapter) covers the places where the LL grammar class pinches; and its modern PEG/packrat branch trades the ambiguity question away entirely for ordered choice plus linear-time memoized backtracking.

## 7 Parsing II: Operator-Precedence Methods

Expressions are where grammars suffer most. Encoding a realistic operator table—C has about fifteen precedence levels—as stratified nonterminals (§2.6) yields a tower of near-identical rules, and a recursive-descent parser for it makes a fifteen-deep chain of calls to parse the literal 7. The operator-precedence family sidesteps the tower entirely: instead of encoding precedence in *grammar structure*, encode it as *numbers in a table* and let a small generic engine consult the table. All three classic algorithms of this family—shunting-yard, precedence climbing, and Pratt parsing—are best understood as one idea in three costumes, and one of them is embedded in virtually every hand-written production parser.

### 7.1 The core question

Linear operator expressions pose exactly one structural question, asked at every operator: *when the parser sits between two operators, as in  $a + b * c$  with the cursor after  $b$ , does  $b$  belong to the operator on its left or on its right?* Precedence answers it when the operators differ ( $*$  wins, so  $b$  goes right); associativity answers it when they tie ( $a - b - c$ : left associativity sends  $b$  left). Give every operator a numeric **binding power** (and a left/right associativity bit), and the answer is a single comparison. Everything below is machinery for asking the question at the right moments.

### 7.2 Dijkstra’s shunting-yard algorithm

The oldest member (Dijkstra, 1961) is a purely iterative, stack-based reordering scheme, named for railway shunting yards. Scan the tokens left to right, maintaining an *operator stack* and an *output*:

- an operand goes straight to the output;
- before pushing an incoming operator  $o$ , first pop to the output every stacked operator with higher precedence than  $o$  (or equal precedence, if  $o$  is left-associative);
- $($  is pushed;  $)$  pops until the matching  $($ ;
- at end of input, pop everything.

If “output” is a list, the result is the expression in *postfix* (reverse Polish) order—history’s favorite compiler-free evaluation format. If instead the output is a stack of *trees*, and “pop operator  $o$  to output” means *pop two trees, combine as  $\text{BinOp}(l, o, r)$ , push the result*, then the algorithm emits the AST directly. The two operations differ only in the output algebra, a first glimpse of a deep pattern (Chapter 11): postfix order *is* a linearized bottom-up tree traversal. Shunting-yard is thus, in spirit, a tiny *bottom-up* parser—it reduces late, once precedence is settled—and indeed it descends from the historically important **operator-precedence parsing** technique of Floyd (1963), a shift-reduce scheme driven by precedence relations between terminals rather than by an LR automaton (Chapter 8).



## 7.3 Precedence climbing

Precedence climbing (Richards 1979; the name due to Clarke) recasts the idea recursively, in a form that drops neatly into a recursive-descent parser. One function does all infix levels; its parameter `minPrec` is a floor: *keep absorbing operators at least this strong; anything weaker belongs to my caller*.

```
function parseExpr(minPrec) : Expr
  left := parseAtom()           - atom: literal, var, (expr), unary
  while lookahead is a binop and prec(lookahead) ≥ minPrec do
    op := consume()
    nextMin := if assoc(op) = LEFT then prec(op) + 1 else prec(op)
    right := parseExpr(nextMin)
    left := BinOp(left, op, right)
  return left
```

The whole theory of the algorithm is in the line computing `nextMin`. Calling the right-hand side with `prec(op) + 1` means the recursion will *refuse* an operator of the same precedence, returning instead—so the loop in the *current* frame captures it, wrapping leftward: left associativity. Calling with `prec(op)` lets an equal-precedence operator be absorbed *inside* the recursion: right associativity (correct for `**` and `=`). Each stack frame owns one precedence plateau; climbing to a stronger operator recurses deeper, and a weaker operator unwinds to whichever frame owns it. A dozen lines replace the whole grammar tower, and adding an operator is a table entry, not a grammar surgery.

## 7.4 Pratt parsing (top-down operator precedence)

Vaughan Pratt’s 1973 method generalizes precedence climbing from “binary infix operators” to *arbitrary token-driven syntax*, and is the form most often found in production hand-written parsers (Douglas Crockford’s JSLint essay revived it; Go’s and many JavaScript engines’ expression parsers are Pratt-shaped). The organizing principle: attach parsing behavior to *tokens*, in two roles.

**nud (null denotation)** — how a token parses when it *begins* an expression: literals and identifiers return leaf nodes; prefix `-` parses its operand at its own (prefix) binding power and returns `Unary`; `(` parses a full expression and expects `)`.

**led (left denotation)** — how a token parses when it appears *after* a completed left subtree, which it receives as an argument: infix `+` builds `BinOp(left, +, parseExpr(bp))`; postfix `!` builds `Factorial(left)` consuming nothing further; `(` *in led position* parses an argument list and builds `Call(left, args)`—function call as “infix parenthesis”; likewise `[` yields indexing and `?` can yield the ternary.

The driver is precedence climbing’s loop with the roles abstracted out:

```
function parseExpr(rbp) : Expr           - rbp = right binding power
  t := consume()
  left := nud(t)
```

```
while rbp < lbp(lookahead) do           - lbp = left binding power
    t := consume()
    left := led(t, left)
return left
```

Every token carries a **left binding power** (lbp); the loop continues while the upcoming token binds tighter to `left` than the context that requested it (`rbp`). Associativity falls out exactly as before, by whether a token's `led` recurses at `lbp` or `lbp-1`. The payoff of the abstraction is uniformity: prefix, infix, postfix, mixfix (`? :`), calls, and indexing all become table entries in one extensible mechanism, which is why Pratt called the technique “top-down operator precedence” and why it composes so cleanly with recursive descent—statements by grammar functions, expressions by the Pratt engine. Conceptually, the trio of algorithms is one algorithm: *shunting-yard makes the operator stack explicit; precedence climbing moves it onto the call stack; Pratt parameterizes what each token does with it.*

## 7.5 Scope and limits

Operator-precedence methods are engines for the *expression layer*, not full-language parsers: they presume a linear operand–operator–operand shape and delegate atoms and statements elsewhere. Within that scope they are unbeatable—linear time, constant-table configurability, and precedence/associativity handled numerically rather than structurally. Their theoretical caveat mirrors their practical power: because precedence lives in a side table rather than in the grammar, the grammar alone no longer determines the tree; the table is now part of your language definition and deserves the same care and documentation.

### Key idea

Stratified grammars encode precedence as *shape*; operator-precedence methods encode it as *numbers* consulted by a ten-line engine. One comparison—incoming binding power vs. current floor—decides every “does *b* go left or right?” question, and choosing  $\geq$  vs.  $>$  at ties is associativity. Pratt parsing is the fully general form: per-token *nud/led* behaviors under one climbing loop, and the de facto standard for expressions in hand-written parsers.

## 8 Parsing III: Bottom-Up Methods — The LR Family

Top-down parsing commits early: it guesses the production *before* seeing what the production derives. Bottom-up parsing commits as late as possible: it reads tokens, gradually recognizes completed right-hand sides in what it has read, and replaces each with its left-hand nonterminal, growing trees from the leaves toward the root. The payoff for the delayed commitment is power—the deterministic bottom-up (LR) grammar class strictly contains the LL class, and in particular digests left recursion natively—at the price that the technique is essentially unwritable by hand and lives inside *parser generators* (Yacc, Bison, and descendants). LR theory, due to Knuth (1965), is the deepest single body of results in parsing; this chapter builds it up in its historical and logical order: shift-reduce, LR(0), SLR(1), LR(1), LALR(1), and GLR.

### 8.1 Shift-reduce parsing and handles

The bottom-up machine keeps a **stack** of grammar symbols (each carrying its subtree) and the remaining input, and repeats two moves:

**shift** — push the next input token onto the stack;

**reduce** — the top of the stack matches the right-hand side of some production  $A \rightarrow \beta$ : pop  $\beta$ , push  $A$  (building the tree node for  $A$  from  $\beta$ 's subtrees).

Acceptance occurs when the stack holds exactly  $S$  and the input is exhausted. Read backwards, the sequence of reductions is a derivation from  $S$ —and because the machine always reduces the leftmost eligible substring, it is precisely a **rightmost derivation in reverse**, the mirror of the top-down story.

**Definition 8.1** (Handle). A **handle** of a right sentential form is the substring matching some production's right-hand side whose reduction is the correct next step of the reverse rightmost derivation.

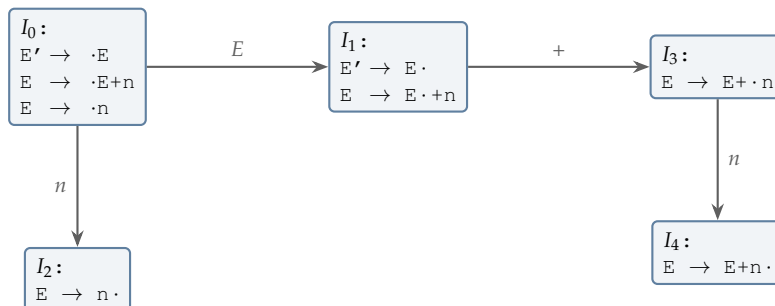
The entire difficulty of bottom-up parsing is *handle finding*: the stack top may match several right-hand sides, or match one that should nonetheless be shifted past. LR theory's central discovery is that the handle question is decidable *by a finite automaton run over the stack*—no search required.

### 8.2 LR(0): items and the characteristic automaton

**Definition 8.2** (LR(0) item). An **LR(0) item** is a production with a dot marking progress:  $A \rightarrow \alpha \cdot \beta$  reads “we may be parsing an  $A$ ; we have seen  $\alpha$ ; we still expect  $\beta$ .”

Items are the states of knowledge available mid-parse. Two operations organize them into an automaton. The **closure** of an item set adds, for every item  $A \rightarrow \alpha \cdot B\beta$  with the dot before nonterminal  $B$ , all items  $B \rightarrow \cdot \gamma$ —if we expect a  $B$  next, we are simultaneously at the start of

every way to build one. The **goto** of a set  $I$  on symbol  $X$  collects each item of  $I$  with the dot before  $X$ , advances the dot, and closes. Starting from the closure of  $S' \rightarrow \cdot S$  (a fresh start symbol) and applying goto exhaustively yields the finite **canonical LR(0) automaton**. For the toy grammar  $E' \rightarrow E, E \rightarrow E + n \mid n$ :



The parser runs this automaton over its stack (states, not symbols, are what is actually stacked): in a state whose items have the dot before a terminal, *shift*; in a state containing a completed item  $A \rightarrow \beta \cdot$ , *reduce* by that production—pop  $|\beta|$  states, then follow the uncovered state’s goto on  $A$ . Here  $I_2$  and  $I_4$  are pure reduce states,  $I_0$  and  $I_3$  pure shift states, and the automaton’s structure alone—zero lookahead—determines every move. Grammars with this property are **LR(0)**; they are rare (note  $I_1$  already mixes a completed item with a shift item, benign only because the completed item is the accept item), which motivates the refinements.

### 8.3 SLR(1), LR(1), LALR(1): the lookahead refinements

When a state contains both a completed item (wanting to reduce) and an item with the dot before a terminal (wanting to shift)—or two completed items—LR(0) is stuck. The refinements differ solely in how cleverly they use *one token of lookahead* to adjudicate, and they form a strict hierarchy of grammar classes:

$$\text{LR}(0) \subset \text{SLR}(1) \subset \text{LALR}(1) \subset \text{LR}(1).$$

**SLR(1)** (DeRemer 1971): keep the LR(0) automaton; permit the reduction  $A \rightarrow \beta \cdot$  only when the lookahead lies in  $\text{FOLLOW}(A)$ , the globally computed follow set of Chapter 6. Simple, and enough for many small languages, but FOLLOW is a *whole-grammar* approximation: it may license a reduction in a state where, in fact, no continuation from *this* context could accept that token.

**LR(1)** (Knuth’s original): make the context exact by baking it into the items. An **LR(1) item**  $[A \rightarrow \alpha \cdot \beta, a]$  carries a lookahead terminal  $a$ : “reduce by  $A \rightarrow \alpha\beta$  only if the next token is  $a$ .” Closure propagates lookaheads precisely ( $B \rightarrow \cdot \gamma$  is added with lookahead  $\text{FIRST}(\beta a)$ ), and the resulting automaton distinguishes states that LR(0) merged. LR(1) is the most powerful deterministic one-token method—Knuth proved LR( $k$ ) grammars capture exactly the deterministic context-free languages (and every LR grammar is unambiguous)—but canonical LR(1) automata were, for decades, considered prohibitively large for real machines.

**LALR(1)** (“lookahead LR”; DeRemer): the engineering compromise that conquered the world. Build (conceptually) the LR(1) automaton, then *merge every pair of states with identical LR(0)*

*cores*, unioning their lookahead sets. The state count collapses back to LR(0)’s size, almost all LR(1) discrimination survives—and this is exactly what `yacc`, `bison`, and their many descendants construct. The merging can manufacture spurious *reduce–reduce* conflicts absent from full LR(1) (never shift–reduce ones), a rare but famously confusing failure mode. Modern algorithms (Pager’s method, IELR) and modern memories have largely dissolved the size argument, and full-LR(1) generators now exist, but “LALR” remains the label on most deployed bottom-up parsers.

## 8.4 Conflicts, and resolving them by declaration

A grammar outside the chosen class announces itself as automaton **conflicts**: a **shift–reduce** conflict (a state may shift the lookahead or reduce) or a **reduce–reduce** conflict (two possible reductions). Conflicts are the generator’s ambiguity-and-beyond diagnostic—necessarily approximate, since true ambiguity is undecidable (§2.5), but concretely actionable. Two celebrated conflicts have canonical resolutions:

- **The dangling else** (§2.7) appears as a shift–reduce conflict on `else`; resolving in favor of *shift* implements nearest-`if` attachment. Yacc-family tools default to shift for exactly this reason.
- **Ambiguous expression grammars.** Rather than stratifying (§2.6), one may feed the generator the naive grammar  $E \rightarrow E + E \mid E * E \mid \dots$  together with **precedence and associativity declarations** (`%left '+'`, `%left '*'` in increasing order). The generator resolves each shift–reduce conflict by comparing the precedence of the production to be reduced against that of the lookahead token—literally the binding-power comparison of Chapter 7, transplanted into the LR automaton. The stratified grammar and the declared-precedence grammar are two encodings of the same operator table.

Undeclared conflicts silently resolved by defaults are a maintenance hazard; the sound practice is to treat unexpected conflicts as errors and either refactor the grammar or add explicit declarations.

## 8.5 Building the AST: the value stack

Where top-down parsers build nodes on the way *down* and return them *up* through calls, an LR parser builds nodes at *reductions*—which is beautifully natural, because a reduction fires exactly when a construct’s constituents are complete. Implementation-wise the parser keeps a **value stack** parallel to the state stack; shifting a token pushes its value (and span); reducing by  $A \rightarrow X_1 \cdots X_k$  pops  $k$  values  $v_1, \dots, v_k$  and pushes the result of that production’s **semantic action**, e.g.

|                          |                                               |
|--------------------------|-----------------------------------------------|
| <code>E : E '+' T</code> | <code>{ \$\$ = BinOp(\$1, Add, \$3); }</code> |
| <code>  T</code>         | <code>{ \$\$ = \$1; }</code>                  |

in Yacc notation, where  $\$i$  names the  $i$ -th popped value and  $$$$  the pushed result. Because reductions occur leaves-first, the AST is assembled strictly *bottom-up*, post-order—each node created only after all its children exist—and left-recursive rules build left-leaning trees with

*no* associativity gymnastics: the grammar’s natural form and the tree’s intended form finally coincide. This mode of tree-building is the archetype of *S-attributed* syntax-directed translation, formalized in Chapter 10.

## 8.6 GLR: bottom-up beyond determinism

When a grammar refuses to fit any deterministic class—C’s infamous  $(a) * b$  (cast or multiplication?) and much of C++ are the standard examples—**generalized LR** (Tomita 1985) keeps the LR machinery but abandons single-mindedness: at a conflict, the parser *does both*, forking. The forks share structure in a **graph-structured stack** (GSS), merging again when their states re-align, so the blow-up is far less than naive parallel simulation; unresolved alternatives yield a **shared packed parse forest** (SPPF) representing all parses compactly, to be disambiguated later by semantic information. GLR runs in linear time on the deterministic portions of the input and degrades toward  $O(n^3)$  only where ambiguity is genuinely present. Bison offers a GLR mode; Elkhound (used by the Elsa C++ parser) demonstrated GLR at industrial scale; and tree-sitter’s error-tolerant incremental parser (Chapter 14) is GLR-derived. GLR completes the bottom-up story: LR for the deterministic heart of a language, GLR as the escape hatch when reality is messier than the hierarchy.

### Key idea

LR parsing is handle-finding by finite automaton: dotted items encode mid-production knowledge, closure/goto compile the grammar into a machine, and one-token refinements ( $SLR \subset LALR \subset LR(1)$ ) trade construction cleverness for grammar coverage. Reductions are the natural moments of AST construction—children first, node after—so left recursion is not merely tolerated but is the *preferred* idiom, and Yacc-style semantic actions on a value stack are how several generations of production front ends were built.

## 9 Parsing IV: General Algorithms, Combinators, and Generators

The deterministic families of the last three chapters cover the grammars that well-designed programming languages use. This chapter completes the survey with the methods that lie outside or across those families: the fully general context-free algorithms (Earley and CYK), which accept *any* grammar including ambiguous ones; parser *combinators*, a way of packaging parsing as an ordinary library; the parser *generators* that industrialize the theory; and, finally, the decision framework for choosing among everything presented.

### 9.1 Earley’s algorithm

Earley’s algorithm (1970) parses *every* context-free grammar—left-recursive, ambiguous,  $\varepsilon$ -ridden, anything—with no grammar preprocessing at all. It is best understood as “all top-down parses at once, with dynamic programming.” The parser sweeps the input once, maintaining for each position  $k$  a **chart** (state set)  $S_k$  of **Earley items**  $(A \rightarrow \alpha \cdot \beta, i)$ , meaning: a parse of  $A$  that *started at position  $i$*  has progressed to the dot by position  $k$ . Three rules saturate the charts:

**Predict** If  $(A \rightarrow \alpha \cdot B\beta, i) \in S_k$ , add  $(B \rightarrow \cdot \gamma, k)$  to  $S_k$  for every production of  $B$  — top-down expectation, exactly LR closure with origin tracking.

**Scan** If  $(A \rightarrow \alpha \cdot a\beta, i) \in S_k$  and the  $k$ -th token is  $a$ , add  $(A \rightarrow \alpha a \cdot \beta, i)$  to  $S_{k+1}$  — consume a terminal.

**Complete** If  $(B \rightarrow \gamma \cdot, i) \in S_k$ , then for every  $(A \rightarrow \alpha \cdot B\beta, j) \in S_i$  add  $(A \rightarrow \alpha B \cdot \beta, j)$  to  $S_k$  — a finished constituent advances everyone who was waiting for it.

The input is accepted iff  $(S' \rightarrow S \cdot, 0) \in S_n$ . Complexity:  $O(n^3)$  in general,  $O(n^2)$  for unambiguous grammars, and *linear* for almost all LR( $k$ ) grammars—so generality costs little when the grammar happens to be tame. Trees (or, under ambiguity, a shared parse forest) are recovered by threading “how did this item get here” back-pointers through completions. Earley shines where the grammar is user-supplied, evolving, or naturally ambiguous: natural-language parsing, grammar prototyping and debugging, and extensible-syntax research systems; refinements (Leo’s optimization for right recursion, the Marpa parser) have made it respectably fast in practice.

### 9.2 CYK

The Cocke–Younger–Kasami algorithm is the other classical general parser, and the most transparent exhibit of parsing-as-dynamic-programming. It requires the grammar in Chomsky Normal Form (every rule  $A \rightarrow BC$  or  $A \rightarrow a$ ; any CFG converts, per Chapter 2) and fills a triangular table:  $T[i, \ell]$  = the set of nonterminals deriving the substring of length  $\ell$  starting at  $i$ . Length-1 entries come from the  $A \rightarrow a$  rules; longer entries by trying every split point:  $A \in T[i, \ell]$  iff some rule  $A \rightarrow BC$  and split  $m$  have  $B \in T[i, m]$  and  $C \in T[i+m, \ell-m]$ . The input parses iff  $S \in T[0, n]$ ; complexity is exactly  $\Theta(n^3 |G|)$ , and back-pointers again recover trees or forests. CYK’s practical



role in compilers is nil—CNF mangles the grammar and hence the trees—but its theoretical role is large: it is the standard constructive proof that CFG recognition is polynomial, the scaffold for probabilistic parsing (PCFGs choose the *most likely* tree by the same table with max-product weights), and the cleanest first algorithm for *proving* things about parsing. Notably, Valiant (1975) reduced CFG parsing to Boolean matrix multiplication, so the true asymptotic frontier of general parsing tracks the matrix-multiplication exponent—a lovely, purely theoretical footnote, since the constants are hopeless.

### 9.3 Parser combinators

Parser combinators change not the algorithm but the *packaging*: a parser becomes a first-class value—a function from input (at a position) to a parse result (value + remaining input, or failure)—and a small algebra of higher-order functions builds big parsers from small ones:

```
type Parser<A> = Input -> Result<(A, Input)>

seq  : Parser<A> -> Parser<B> -> Parser<(A,B)>      - then
alt  : Parser<A> -> Parser<A> -> Parser<A>          - or else
many : Parser<A> -> Parser<List<A>>                - zero or more
map  : (A -> B) -> Parser<A> -> Parser<B>          - build AST nodes here

expr = map(buildBinOp, seq(term, many(seq(plusTok, term))))
```

The grammar is now an executable expression in the host language; `map` is where AST constructors attach, so the combinator program reads like an EBNF grammar annotated with semantic actions. Theoretically, standard combinator libraries implement recursive descent with backtracking—so everything from Chapter 6 transfers: left recursion loops forever unless the library detects it, naive `alt` backtracking can go exponential, and industrial libraries (Parsec and its many descendants) therefore commit to the first alternative that *consumes input*, restoring predictive behavior, good error messages, and near-linear performance at the cost of requiring explicit backtracking annotations. The deep observation, due to the functional-programming tradition, is that `Parser` forms a *monad*: `seq`-with-binding lets a later parser depend on an earlier parser’s *value*, which quietly exceeds context-free power (one can parse `n` followed by exactly `n` items). Combinators are thus the most *compositional* and host-integrated of all approaches—ideal for DSLs, protocols, and rapidly evolving syntaxes—while remaining, underneath, our familiar top-down theory.

### 9.4 Parser generators and industrial tools

A **parser generator** is a compiler whose source language is grammars and whose target is parsers; it is where this document’s theory is productized. The landscape, keyed to the theory chapters:



| Tool             | Theory             | Lookahead    | Notes for AST builders                                                                               |
|------------------|--------------------|--------------|------------------------------------------------------------------------------------------------------|
| Yacc / GNU Bison | LALR(1); GLR opt.  | 1 token      | Semantic actions on value stack (§8.4); <code>%left/%right</code> precedence; C/C++/more.            |
| ANTLR 4          | ALL(*) adaptive LL | unbounded    | Generates recursive descent + auto parse trees; visitors/listeners generated; many host languages.   |
| Menhir (OCaml)   | full LR(1)         | 1 token      | Typed semantic actions; excellent conflict explanations.                                             |
| Lemon (SQLite)   | LALR(1)            | 1 token      | Re-entrant, pull-based; powers SQLite’s SQL front end.                                               |
| tree-sitter      | GLR-based, incr.   | —            | Error-tolerant lossless CSTs for editors; C runtime, grammars in JS DSL (Ch. 14).                    |
| PEG tools        | PEG / packrat      | unbounded    | pest, PEG.js, CPython’s; no separate lexer; no ambiguity by fiat; watch alternative ordering (§6.5). |
| Combinator libs  | recursive descent  | configurable | Parsec lineage; nom, chumsky, etc.; grammar = host-language program (§9.3).                          |

Two structural observations. First, generators split on *what tree you get*: Yacc-family tools build nothing until your actions do (total freedom, total responsibility), while ANTLR and tree-sitter emit an automatic (homogeneous) parse tree that you then walk or convert—quicker to start, one more representation to manage (§4.3). Second, the hand-written/generated pendulum is empirical, not ideological: generated parsers dominate when the grammar is the product (SQL dialects, configuration languages, research prototypes) or must be verifiably in a class; hand-written recursive-descent-plus-Pratt dominates production compilers (GCC, Clang, Go, Rust, TypeScript, V8) because error *recovery*, error *messages*, and IDE integration—Chapter 14’s topics—are easier to craft in ordinary code than to encode in a generator’s action language.

## 9.5 Choosing a method

The consolidated map of every method in this survey:

| Method                 | Grammar class      | Time                      | Builds tree   | Hallmark                    |
|------------------------|--------------------|---------------------------|---------------|-----------------------------|
| Recursive descent      | LL( <i>k</i> )-ish | $O(n)$                    | top-down      | readable, hand-craftable    |
| Table LL(1)            | LL(1)              | $O(n)$                    | top-down      | table-driven, tiny          |
| PEG + packrat          | PEGs               | $O(n)$                    | top-down      | ordered choice, scannerless |
| Pratt / prec. climbing | operator exprs     | $O(n)$                    | bottom-up-ish | table-driven precedence     |
| SLR / LALR / LR(1)     | DCFL subclasses    | $O(n)$                    | bottom-up     | generators, left recursion  |
| GLR                    | all CFGs           | $O(n) \rightarrow O(n^3)$ | forest        | forks on conflict           |
| Earley                 | all CFGs           | $O(n) \rightarrow O(n^3)$ | forest        | no preprocessing            |
| CYK                    | all CFGs (CNF)     | $\Theta(n^3)$             | forest        | DP; PCFG-ready              |
| Combinators            | host-defined       | varies                    | top-down      | compositional library       |

And the decision procedure, distilled:

1. **Building a real language front end, want control and great errors?** Hand-written recursive descent for statements + Pratt for expressions. This is the modern production default.
2. **Grammar-as-specification, want machine-checked determinism?** An LR-family generator (Bison, Menhir); embrace left recursion and `%left` declarations.
3. **Fast iteration, complex token-level context, dislike lexer/parser split?** PEG/packrat.
4. **Editor tooling over possibly-broken code, incremental updates?** tree-sitter (Chapter 14).

5. **Genuinely ambiguous or user-supplied grammars, or research?** GLR or Earley, with forest post-disambiguation.
6. **Small DSL inside a host application?** Parser combinators.

**Key idea**

Beyond the deterministic families lies a complete toolbox: Earley and CYK parse *anything* at polynomial cost and return forests when grammars are ambiguous; combinators re-express top-down parsing as a compositional library; and generators industrialize each theory. Choosing a method is choosing a point in a four-way trade among grammar freedom, performance guarantees, error-handling craft, and tooling—and the production world’s revealed preference is hand-written top-down for compilers, LR generators for grammar-first projects, and GLR-style engines for editors.

# 10 Building the Tree: Syntax-Directed Translation and Attribute Grammars

Every parsing chapter so far has smuggled in tree construction through phrases like “semantic action” and “return a node.” This chapter surfaces the general theory behind those phrases. The framework—**syntax-directed translation**, whose formal backbone is Knuth’s **attribute grammars** (1968)—explains uniformly *how any computation, AST construction chief among them, can be attached to a grammar and scheduled by a parse*. It also settles the practical questions the earlier chapters raised: why LR parsers and AST-building agree so naturally, why LL parsers need extra care, and what exactly the recursive-descent associativity fix of §6.3.1 was an instance of.

## 10.1 Syntax-directed definitions

**Definition 10.1** (Syntax-directed definition). A **syntax-directed definition** (SDD) associates with each grammar symbol a set of **attributes** (typed values) and with each production a set of **semantic rules** defining some attributes in terms of others within that production’s local tree neighborhood.

Conceptually: take a parse tree, hang variables (attributes) on its nodes, and let each production’s rules say how neighboring variables determine one another. Running the rules to completion *evaluates* the tree—and if the distinguished attribute being computed is “the AST node for this construct,” evaluation *is* AST construction. Knuth’s decisive classification splits attributes by the direction information flows:

**Synthesized attributes** flow *upward*: an attribute of the left-hand-side nonterminal is defined from attributes of the right-hand-side symbols (the node’s children). “The value of  $E$ ” and “the AST of  $E$ ” are synthesized.

**Inherited attributes** flow *downward or sideways*: an attribute of a right-hand-side symbol is defined from the left-hand side’s attributes or from siblings. “The current declared type,” “the enclosing scope,” and “the tree accumulated so far” (as we will see) are inherited.

An SDD using only synthesized attributes is **S-attributed**. An SDD is **L-attributed** if every inherited attribute of a right-hand-side symbol depends only on attributes of symbols *to its left* in the production (plus the parent’s inherited attributes)—i.e., information never flows right-to-left. These two classes are exactly the ones parsers can evaluate *on the fly*, and they map one-to-one onto the two great parsing families.

## 10.2 S-attributed SDDs and bottom-up parsing

AST construction in its purest form is an S-attributed definition—each node from its children:

|                         |                                                    |
|-------------------------|----------------------------------------------------|
| $E \rightarrow E_1 + T$ | $E.ast = \text{BinOp}(E_1.ast, \text{Add}, T.ast)$ |
| $E \rightarrow T$       | $E.ast = T.ast$                                    |
| $T \rightarrow T_1 * F$ | $T.ast = \text{BinOp}(T_1.ast, \text{Mul}, F.ast)$ |
| $F \rightarrow ( E )$   | $F.ast = E.ast$                                    |

|                            |                                               |
|----------------------------|-----------------------------------------------|
| $F \rightarrow \text{num}$ | $F.\text{ast} = \text{Num}(\text{num.value})$ |
|----------------------------|-----------------------------------------------|

Note the two abstraction moves happening *in the rules*: unit productions copy the child’s tree (collapsing CST chains), and the parenthesis rule forwards the inner tree (deleting notation)—the abstraction function of Chapter 3, expressed as semantic rules. The scheduling theorem is immediate: *an S-attributed SDD can be evaluated by any bottom-up parse*, because an LR reduction fires exactly when a production’s children are complete, which is exactly when its synthesized rules are ready. Yacc’s value stack (§8.4) is nothing but the runtime of S-attributed evaluation; the notation  $\$ \$ = f(\$1, \dots, \$k)$  is the semantic rule. Span computation rides along synthesizedly: a node’s span is the hull of its children’s spans, assembled at each reduction.

### 10.3 L-attributed SDDs and top-down parsing

Top-down parsers run rules *during the descent*, so they can also thread information downward—inherited attributes—provided nothing depends on material not yet parsed: precisely the L-attributed condition. The implementation dictionary is direct: *inherited attributes are parameters of the recursive parse functions; synthesized attributes are their return values*. And with that dictionary, the associativity fix of §6.3.1 is finally seen for what it is. The transformed grammar  $E \rightarrow T E'$ ,  $E' \rightarrow + T E' \mid \varepsilon$  builds the correct left-leaning tree via one inherited and one synthesized attribute on  $E'$ :

|                              |                                                                           |                                |
|------------------------------|---------------------------------------------------------------------------|--------------------------------|
| $E \rightarrow T E'$         | $E'.\text{inh} = T.\text{ast}$                                            | $E.\text{ast} = E'.\text{syn}$ |
| $E' \rightarrow + T E'_1$    | $E'_1.\text{inh} = \text{BinOp}(E'.\text{inh}, \text{Add}, T.\text{ast})$ |                                |
|                              | $E'.\text{syn} = E'_1.\text{syn}$                                         |                                |
| $E' \rightarrow \varepsilon$ | $E'.\text{syn} = E'.\text{inh}$                                           |                                |

The accumulated left context is *inherited* down the  $E'$  spine—each step wraps it as the new left operand—and the finished tree is synthesized back at the  $\varepsilon$ . The `parseE` loop of Chapter 6, with its `left` variable, is the compiled form of exactly this SDD: `left` is  $E'.\text{inh}$  living in a loop variable, and the function’s return value is the synthesized result. What looked like an ad hoc trick is a theorem: *L-attributed definitions over LL grammars are evaluable in one predictive pass*, with the tail-recursive spine optimized into iteration.

### 10.4 Attribute grammars in general

Freed from parser-scheduling constraints, attribute grammars are a full declarative language for tree computation. A general AG may mix synthesized and inherited attributes arbitrarily; evaluation then requires building the **dependency graph** among attribute instances on the given tree and processing it in topological order. The key results: a *circular* AG (some tree yields a cyclic dependency graph) is erroneous, and circularity is decidable—Knuth’s test, exponential in the worst case but fine in practice—while for the well-behaved **ordered** subclasses a static evaluation schedule can be precomputed per production. This generality is precisely what post-parse *semantic analysis* needs: symbol tables flow down (inherited), types flow up (synthesized), and Chapter 12’s name resolution and type checking are, in classical terms, one big attribute grammar over the

AST—whether or not one uses an AG *system* (Eli, JastAdd, Silver) or hand-writes the traversals it would have generated.

## 10.5 Translation schemes and marker nonterminals

An SDD says *what* to compute; a **syntax-directed translation scheme** (SDT) additionally pins *when*, by embedding actions at positions inside right-hand sides:  $A \rightarrow \alpha \{ act \} \beta$  runs *act* after parsing  $\alpha$ , before  $\beta$ . Top-down parsers execute embedded actions trivially (they are statements between calls). Bottom-up parsers only have a natural hook at reductions (end of production); generators simulate mid-rule actions by silently rewriting  $\{act\}$  into a fresh **marker nonterminal**  $M \rightarrow \varepsilon$  whose sole reduction runs the action—elegant, but the inserted  $\varepsilon$ -production can perturb the grammar’s LALR-ness, a classic gotcha when porting grammars between tools.

## 10.6 The payoff for AST builders

Assembling the chapter’s results into working advice:

- **Design tree construction as an S-attributed definition whenever possible:** node from children, span as hull of child spans. It runs unchanged on any parser, and it is what Yacc actions, combinator `maps`, and recursive-descent return values all implement.
- **Reach for inherited attributes (parameters) exactly where left context matters:** left-associative folds under LL grammars, contextual flags (“are we inside a loop?”), and propagating environment-like data during any later tree walk.
- **Treat the parser/AST boundary as a translation, not an identity:** the grammar the algorithm needs (left-factored, recursion-eliminated, marker-ridden) and the tree the language means are different artifacts, and the semantic rules are the officially sanctioned place where the difference is reconciled.

### Key idea

Attribute grammars are the general theory of “computation attached to syntax.” Two scheduling theorems organize all of practice: synthesized-only definitions run on bottom-up parses (Yacc’s value stack), and L-attributed definitions run on top-down parses (parameters down, return values up). AST construction is the canonical S-attributed computation—and every trick in Chapters 6–9 for building the right tree is a disguised instance of these theorems.

# 11 Consuming the Tree: Traversal, Visitors, and Folds

Once the parser has produced an AST, every subsequent activity—interpretation, type checking, optimization, printing, linting—is a *computation over the tree*. This chapter develops the small but load-bearing theory of such computations: traversal orders, the visitor pattern (the OO world’s standard answer to the expression problem of §4.4), and the unifying mathematical view of tree computations as *folds*.

## 11.1 Traversal orders

A traversal is a discipline for visiting every node exactly once. For rooted ordered trees the canonical orders are:

**Pre-order** (node, then children): the natural order for *top-down* information—establishing context before descending. Scope entry, inherited-attribute propagation, and code generators that emit prologues use pre-order moments. Note that a top-down parser *creates* nodes in pre-order.

**Post-order** (children, then node): the natural order for *bottom-up* synthesis—a node’s result from its children’s results. Evaluation, type synthesis, constant folding, and code generators that need operands before operators are post-order computations. A bottom-up parser creates nodes in post-order, which is why LR parsing and S-attributed construction fit hand in glove (§8.4, Chapter 10).

**In-order** (left child, node, right child): meaningful only for binary-ish nodes; walking a `BinOp` tree in-order recovers the infix reading, which is essentially what a pretty-printer does (plus minimal parentheses where a child’s operator binds looser than its parent’s).

Most real passes are not purely one order but interleave a pre-order “enter” moment and a post-order “exit” moment around the recursive descent into children—exactly the `enter/exit` callback pairs exposed by traversal APIs such as Babel’s and ESLint’s, and exactly the shape of an attribute grammar with both inherited (enter) and synthesized (exit) attributes.

## 11.2 The visitor pattern

In an OO encoding (§4.3), a pass “switch on the node’s class” has two mechanical realizations. The naive one—chains of type tests—is slow and unchecked. The classical one is the **visitor pattern**: the pass becomes an object with one method per node class, and dispatch is delegated to the nodes themselves via **double dispatch**:

```
interface Visitor<R>
  R visitNum(Num n);  R visitVarRef(VarRef v);  R visitBinOp(BinOp b);

abstract class Expr { abstract <R> R accept(Visitor<R> v); }
```

```

class BinOp extends Expr {
  Expr left; Op op; Expr right;
  - the second dispatch:
  <R> R accept(Visitor<R> v) { return v.visitBinOp(this); }
}

class Evaluator implements Visitor<Value> {
  Value visitNum(Num n) { return IntVal(n.value); }
  Value visitBinOp(BinOp b) {
    l := b.left.accept(this); r := b.right.accept(this);
    return apply(b.op, l, r);
  }
}

```

The call `expr.accept(v)` dispatches first on the node’s dynamic class (choosing which `accept` runs) and then on the visitor (choosing which `visitX` runs)—simulating, in single-dispatch languages, the pattern-matching that ADT languages provide natively. The design consequences were previewed in §4.4: visitors make *adding a pass* a single new class and *adding a node kind* a change to every visitor—the right trade for compilers, whose node set freezes early while passes accumulate forever. Standard refinements: a *default/base visitor* supplying generic child-recursion so concrete passes override only the nodes they care about; *listener* style (void enter/exit callbacks with the framework driving traversal, as in ANTLR) versus *visitor* style (the pass drives recursion and returns values); and visitors parameterized by return and context types, which are precisely synthesized and inherited attributes wearing OO clothes.

### 11.3 Folds: the mathematical shape of every pass

The functional view subsumes all of the above in one construction. Recall from §3.3 that an AST type is an initial algebra—the least set generated by its constructors. The corresponding recursion principle:

**Definition 11.1** (Fold / catamorphism). Fix result type  $R$  and one function per constructor— $f_{\text{Num}} : \mathbb{Z} \rightarrow R$ ,  $f_{\text{Var}} : \text{Sym} \rightarrow R$ ,  $f_{\text{Bin}} : R \times \text{Op} \times R \rightarrow R$  (an **algebra** for the signature). The **fold** determined by the algebra is the unique function  $\text{fold} : \text{Expr} \rightarrow R$  satisfying

$$\text{fold}(\text{Num } n) = f_{\text{Num}}(n) \quad \text{fold}(\text{BinOp}(l, \odot, r)) = f_{\text{Bin}}(\text{fold } l, \odot, \text{fold } r)$$

and similarly for each constructor: replace every constructor by its algebra function, bottom-up.

Uniqueness (initiality) is what makes this more than notation: *to define a structurally recursive pass is exactly to choose an algebra*, and two passes are equal iff their algebras are. The evaluator is the fold whose algebra interprets `Num` as its integer and `BinOp` as arithmetic; the pretty-printer is the fold into strings; “count the nodes” is the fold into  $\mathbb{N}$  with  $f_{\text{Bin}}(l, \_, r) = l + r + 1$ ; type synthesis is a fold into types (Chapter 12); and a visitor with return type  $R$  is an algebra, its `visitX` methods the components, with `accept` playing `fold`. Even the parser fits: Chapter 10’s S-attributed construction is the fold over the (virtual) parse tree whose algebra is the semantic actions—which is why shunting-yard could emit either RPN or ASTs by “changing the output

algebra” (Chapter 7). Passes needing downward context are folds whose result type is a *function* of the context,  $R = Env \rightarrow A$ —the fold/attribute-grammar correspondence again.

Two practical corollaries. First, *structural induction* is the proof principle dual to fold: to prove  $P$  holds of every AST, prove it for each constructor assuming it for the children—the standard way one verifies “the optimizer preserves semantics” or “the printer–parser round-trips.” Second, generic-traversal libraries (“visit every node, applying this rewrite where it matches”—Babel’s path visitors, Roslyn’s `CSharpSyntaxRewriter`, strategy libraries like Stratego) are reusable fold skeletons; a **rewrite pass** is the special fold whose algebra rebuilds each node unchanged except where a rule fires, and under the persistence discipline of §4.5 the untouched subtrees are *shared*, making whole-tree rewrites cheap.

#### Pitfall

Deeply nested source (machine-generated code, huge literal arrays) can overflow the host call stack of any naively recursive traversal—and of the recursive-descent parser that built the tree. Production tools either impose nesting limits (compilers commonly do), convert traversals to an explicit-stack loop, or use continuation/trampoline techniques. The fold view makes the conversion mechanical: the explicit stack holds pending algebra applications.

#### Key idea

There is exactly one deep idea in tree consumption: *replace constructors by functions*. Choose an algebra and the fold does the rest; visitors are algebras for OO hosts; enter/exit callbacks are the inherited/synthesized moments; and every pass in the remaining chapters—resolution, checking, desugaring, lowering, printing—is a fold with a particular algebra.



## 12 Semantic Analysis: Decorating the Tree

The parser guarantees the program is *well-formed*; it cannot guarantee the program *makes sense*. “Every name refers to something,” “operands have compatible types,” “`break` appears inside a loop”—these are the context-sensitive conditions deliberately excluded from the grammar in Chapter 2. **Semantic analysis** is the family of AST passes that checks them and, in doing so, *decorates* the tree with the information every later stage needs. Formally, the whole enterprise is one large attribute grammar (Chapter 10) evaluated by the traversals of Chapter 11; this chapter gives its three pillars—symbol tables, name resolution, and type checking—their own treatment.

### 12.1 Symbol tables and scopes

**Definition 12.1** (Symbol table, scope). A **symbol table** maps names to their **symbols**—records describing what a name denotes (kind: variable/function/type/...; type; definition site; attributes such as mutability). A **scope** is a region of the program with its own table; scopes nest, forming a tree parallel to (and induced by) the AST’s block structure.

The operative structure is the **environment**: a chain of tables from the current scope outward to the global one. Lookup walks the chain inward-out, implementing **lexical (static) scoping**: the innermost declaration wins, and *shadowing* an outer name is legal precisely because lookup stops at the first hit. During a traversal the environment behaves like a stack—push a fresh table at each scope-introducing node (block, function, class), pop on exit—which identifies it as an *inherited attribute* flowing down the walk. Closures complicate the picture beautifully: a function value may outlive the activation that created its enclosing scope, so environments captured by closures cannot live on a stack; they become heap-allocated and linked by parent pointers.

### 12.2 Name resolution

**Name resolution** (binding analysis) connects every *use* of a name to its *declaration*: a walk that maintains the environment as above and, at each `Var/Name` node, records the outcome. The recorded link—a pointer to the symbol, stored in the node or in a side table (§4.5)—turns the AST’s textual names into a graph of actual references, and downstream passes never do string lookup again. The classical subtleties are policy questions your language must answer and your walk must implement: use-before-definition (C says no; Python resolves at call time; mutually recursive functions require a pre-pass that declares all signatures first); whether one identifier space or several *namespaces* exist (functions vs. variables vs. types); and re-declaration rules. CPython’s `ast` module makes the input side of this visible: `Name` nodes carry a `ctx` of `Load`, `Store`, or `Del`, distinguishing uses from bindings syntactically before resolution even begins. Diagnostics for failures—undeclared name, duplicate declaration—are where the source spans threaded since Chapter 4 pay their keep.

## 12.3 Type checking

Type checking decides whether every operation receives operands it is defined on. Its theory is presented as an *inference system*: judgments  $\Gamma \vdash e : \tau$  (“in typing environment  $\Gamma$ , expression  $e$  has type  $\tau$ ”) derived by one rule per node kind—literally an S-attributed definition over the AST, with  $\Gamma$  inherited and  $\tau$  synthesized:

$$\frac{n \text{ an integer literal}}{\Gamma \vdash \text{Num}(n) : \text{Int}} \quad \frac{x:\tau \in \Gamma}{\Gamma \vdash \text{Var}(x) : \tau} \quad \frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash \text{BinOp}(e_1, +, e_2) : \text{Int}}$$

$$\frac{\Gamma, x:\tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{Lambda}(x:\tau_1, e) : \tau_1 \rightarrow \tau_2} \quad \frac{\Gamma \vdash f : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash a : \tau_1}{\Gamma \vdash \text{Call}(f, a) : \tau_2}$$

Reading the rules as code: each is a case of a post-order fold whose enter-moment extends  $\Gamma$  (the lambda rule) and whose exit-moment synthesizes the type—and a type *error* is a node where no rule applies. Everything beyond this core is enrichment along known axes: *coercions* (inserting implicit conversion nodes, i.e., the checker *rewrites* the tree, not just reads it); *overloading* (the operator’s meaning chosen from operand types, recorded in the node for codegen); *subtyping* ( $\tau \leq \tau'$  side conditions); and full **type inference** (Hindley–Milner), where types may be omitted and the walk instead generates equality *constraints* between type variables, solved by **unification**—the same fold, synthesizing constraints rather than verdicts. Statement-level checks ride along as a second family of judgments (“well-formed in context”): `break` legal only under a loop flag (an inherited boolean), `return` matching the enclosing function’s declared type, definite-assignment analyses, and so on.

## 12.4 The decorated tree

The output of semantic analysis is the same tree, *knowing more*: names linked to symbols, expressions labeled with types, implicit operations made explicit. The engineering options for “knowing more” were laid out in §4.5—mutate fields, keep side tables keyed by node identity, or rebuild into a distinct *typed AST* whose very type signature guarantees the invariants (in expressive hosts, an indexed/GADT family where `Expr Int` and `Expr Bool` are different types, making ill-typed trees unrepresentable). Whichever strategy, the decorated AST is the true interface between front end and back end: Chapter 13 consumes exactly this artifact.

### Key idea

Semantic analysis is the attribute grammar the CFG could not express: environments and expectations flow *down* (inherited), resolutions and types flow *up* (synthesized), and the tree ends the phase decorated with links and labels. The inference-rule notation  $\Gamma \vdash e : \tau$  is not decoration on the theory—it *is* the pass, one rule per node kind, executed as a fold.

## 13 After the AST: Desugaring, Lowering, and Interpretation

The decorated AST of Chapter 12 is the summit of the front end. This chapter descends the other side: the three standard fates of a finished tree—being *simplified* (desugaring), being *translated* into lower-level representations (lowering), or being *executed* directly (tree-walking interpretation). Each is a fold with a particular algebra (Chapter 11); what differs is the target.

### 13.1 Desugaring: AST-to-AST simplification

Languages are full of **syntactic sugar**—convenient surface forms definable in terms of a smaller core. **Desugaring** is the rewrite pass that performs the definitions: `a += b` becomes `a = a + b`; a `for` loop becomes initialization plus a `while`; list comprehensions become loops with appends; `async/await` famously becomes a state machine. The theoretical payoff is concentrated in the notion of a **core language**: if every sugared node rewrites into a small trusted kernel, then semantics, type soundness, and the entire back end need only be defined for the kernel—the strategy taken to its logical conclusion by languages like Haskell, whose vast surface desugars into a tiny explicitly-typed core calculus that is independently type-checked as a compiler self-test. Design questions with real consequences: *when* to desugar (parse time keeps the pipeline short but wrecks error messages and tooling, which see code the user never wrote—the fidelity concerns of §3.5; a separate post-parse pass is the conventional answer); whether the target is the *same* AST type (a rewrite fold) or a genuinely smaller core type (a translation fold, letting the host type system prove no sugar survives); and **hygiene**, the capture-avoidance discipline required whenever a rewrite invents temporary variables—the same problem Lisp macros and any user-facing AST-rewriting system (Babel plugins, Rust macros) must solve.

### 13.2 Lowering: leaving the tree

Trees are the right shape for syntax and checking, but the wrong shape for two later jobs: expressing *control flow as a graph* (optimizers want to see that a loop is a cycle) and expressing *linear execution* (machines run instruction sequences). **Lowering** translates the AST into successively flatter **intermediate representations** (IRs):

**Linearization / flattening.** A post-order fold emits, for each node, instructions computing its value into a fresh temporary: `BinOp(l, +, r)` becomes *code for l* `||` *code for r* `||` `t3 := t1 + t2`. The result is **three-address code**; the fold’s algebra returns (code, result-temporary) pairs. Control constructs lower to conditional and unconditional jumps between generated labels—the moment tree structure officially becomes graph structure.

**Stack-machine code.** The even simpler target: post-order emission where `BinOp` becomes *emit l*; *emit r*; `ADD`. That this works is exactly the shunting-yard observation of Chapter 7: postfix is linearized post-order. CPython’s compiler is the canonical live example—source  $\rightarrow$  AST (visible via `ast`)  $\rightarrow$  control-flow graph  $\rightarrow$  bytecode for a stack VM (visible via `dis`).

**CFG and SSA.** Serious optimizers reorganize the linear code into a **control-flow graph** of basic blocks and then into **static single assignment** form (each variable assigned once; join points mediated by  $\phi$ -functions), the fixed point of decades of optimization research and the core of LLVM IR. The AST’s job is to be *easily* translatable into this world—one reason ASTs stay close to source structure rather than trying to be optimization-friendly themselves.

The governing principle across all lowering: *each representation exists to make one class of questions easy*—the AST for “what did the programmer say,” the CFG for “how does control flow,” SSA for “where do values come from”—and a compiler is a pipeline of such representations connected by folds.

### 13.3 Tree-walking interpretation

The most direct fate: *execute the AST itself*. A tree-walking interpreter is the evaluation fold made operational—environments inherited, values synthesized:

```
function eval(node, env) : Value
  match node with
  | Num(n)           -> IntVal(n)
  | VarRef(x)        -> lookup(env, x)
  | BinOp(l, op, r)  -> apply(op, eval(l, env), eval(r, env))
  | If(c, t, f)      -> if truthful(eval(c, env))
                        then eval(t, env) else eval(f, env)
  | Lambda(p, body)  -> Closure(p, body, env)      - captures env!
  | Call(f, a)       -> let Closure(p, body, cenv) = eval(f, env) in
                        eval(body, extend(cenv, p, eval(a, env)))
```

Its theoretical pedigree is exact: this function *is* a **big-step (natural) operational semantics**, the judgment  $\langle e, env \rangle \Downarrow v$  with one inference rule per constructor—write the rules, transcribe them into `match` arms, and you have both a formal semantics and its reference implementation, which is why language specifications and their first interpreters so often co-evolve. Statements execute by a sibling function threading state; control-flow escape (`return`, `break`, exceptions) is modeled either by exceptions in the host or by returning explicit control signals. Performance is the known cost—every node dispatch and environment lookup paid at runtime, typically one to two orders of magnitude off compiled code—and the standard escalation path is exactly the previous section: lower to bytecode for a VM, then JIT. But as *Crafting Interpreters* demonstrates by building one of each, the tree-walker is unbeatable for clarity, for prototyping a language’s semantics, and as the executable definition against which faster implementations are tested.

#### Key idea

Downstream of the AST, everything is translation between representations, and every translation is a fold: desugaring folds the tree into a smaller tree (surface  $\rightarrow$  core), lowering folds it into sequences and graphs (tree  $\rightarrow$  TAC/bytecode  $\rightarrow$  CFG/SSA), and interpretation folds it into values *right now*—the executable transcription of the language’s big-step semantics.

## 14 Advanced Topics: Errors, Fidelity, and Incrementality

Classical parsing theory quietly assumes a benevolent world: the input is complete, correct, and parsed once. Real tools live elsewhere—users mistype, editors reparse on every keystroke, and formatters must reproduce the very trivia the theory taught us to discard. This chapter covers the extensions that close the gap: syntax error handling and recovery, full-fidelity (lossless) trees and the red–green architecture, incremental parsing, and the tooling ecosystem these ideas enable.

### 14.1 Syntax errors: detection, reporting, recovery

Deterministic parsers enjoy the **viable-prefix property**: LL and LR parsers detect an error at the *first* token that cannot extend any valid program—the earliest theoretically possible moment, with the automaton’s state encoding exactly what *would* have been legal (the basis of “expected one of ...” messages, and of the human-written diagnostics that are the chief reason production compilers hand-write their parsers, §9.4). The hard problem is what to do *next*: a compiler that stops at the first error is miserable to use, so the parser must **recover** and continue, ideally reporting each true error once while inventing none. The classical repertoire:

**Panic mode.** On error, discard tokens until reaching a member of a **synchronization set**—tokens that reliably begin or end constructs (`;`, `}`, statement keywords)—then resume. Simple, unkillable, and the backbone of real recovery; the craft is choosing sync sets per context (in recursive descent: catch in the statement loop, synchronize, continue), and suppressing the *cascade* of spurious errors immediately after resumption.

**Phrase-level repair.** Locally insert or delete a token (the missing `;`, the unbalanced `)`) and continue. More precise, riskier: a wrong guess manufactures phantom errors downstream. Modern refinements search for a minimum-cost repair sequence within a bounded window.

**Error productions.** Extend the grammar with rules matching *known* mistakes (e.g., `if` without parentheses) so the “error” parses successfully into a diagnostic-bearing node with a tailored message. Yacc/Bison’s `error` token institutionalizes the idea: productions like `stmt : error ';' let the LR machinery pop to a state where the error rule applies and resynchronize on the ;.`

**Error nodes in the AST.** The modern synthesis: recovery is not just about the *parser* continuing but about the *tree* existing anyway. IDE-grade parsers guarantee a complete tree for arbitrarily broken input, with explicit `Error`/`Missing` nodes marking the damage—so that highlighting, completion, and analysis of the *valid* 99% of the file proceed undisturbed. Designing node types to admit “holes” is thus a first-class AST design concern, not an afterthought.

## 14.2 Full-fidelity trees and the red–green architecture

Chapter 3 set the abstraction dial; tools that *write source back* (formatters, refactorers, codemods) need it at the lossless end: every token, every space, every comment, recoverable byte-for-byte. The engineering problem is doing this without sacrificing the abstract view or the ability to edit cheaply, and the canonical solution is Roslyn’s (the C#/VB compiler platform) **red–green tree**:

- The **green tree** is the persistent core: immutable nodes storing kind, children, and *widths* (text lengths) but—crucially—*no absolute positions and no parent pointers*. Position-freedom is what makes green nodes *relocatable and shareable*: an edit rebuilds only the spine from the change to the root ( $O(\text{depth})$  new nodes, everything else shared), the persistence payoff promised in §4.5. Trivia (whitespace, comments) hangs off tokens as leading/trailing lists, so the green tree’s concatenated text *is* the file.
- The **red tree** is a lazily materialized facade over the green tree that adds what the green tree denied itself: absolute positions (computed by summing widths on the way down) and parent pointers. Red nodes are created on demand during traversal and discarded freely; “red” and “green” are simply the colors the Roslyn designers happened to draw them in.

The two-layer scheme reconciles all the conflicting requirements at once—losslessness, immutability with cheap edits, parent/position convenience for analyzers—and has been widely imitated (Swift’s SwiftSyntax, Rust-analyzer’s Rowan). In the Python world, **LibCST** occupies the same niche with one representation: a concrete-syntax tree that round-trips exactly, exposed through an AST-like typed API—the tool of choice for large-scale codemods where `ast` (which discards trivia) cannot serve.

## 14.3 Incremental parsing

An IDE reparsing a megabyte file on every keystroke needs better than  $O(n)$  per edit; it needs cost proportional to the *change*. **Incremental parsing** achieves this by reusing subtrees of the previous parse that provably cannot have been affected. The modern exemplar is **tree-sitter** (Max Brunsfeld, GitHub), which combines the themes of this chapter: a generated **GLR-based** parser (§8.4ff.) producing *lossless concrete* trees, with aggressive *error recovery* guaranteeing a tree for any input, and *incremental* reparse—after an edit, unchanged subtrees from the old tree are reused wholesale (their positions shifted), and only the damaged region is re-read, following the line of work begun by Wagner and Graham’s incremental LR parsing. The persistence discipline is again what makes reuse safe: subtrees are immutable values, so sharing them between the old and new tree is sound. Tree-sitter’s query language (S-expression patterns over the tree) then powers highlighting and structural navigation across hundreds of editor language integrations—parsing theory, shipped to every keystroke you type.

## 14.4 The AST tooling ecosystem

The ideas above underwrite the tool categories from Chapter 1, now explainable precisely:

**Linters** (ESLint, Pylint, Clang-Tidy): rule = visitor over node kinds (ESLint rules literally register

callbacks keyed by ESTree node types, with *enter/exit* moments per §11); autofix = span-guided text edit or tree rewrite.

**Transpilers & codemods** (Babel, TypeScript, jscodeshift, LibCST codemods): parse → rewrite fold → print; hygiene and fidelity per Chapters 13 and this one.

**Formatters** (Prettier, Black, clang-format): parse to (at least comment-preserving) tree, then *pretty-print*: a fold into a document algebra of groups and breakpoints (the Wadler/Oppen line of pretty-printing theory) rather than into strings—which is why formatters produce optimal line breaks instead of taped-together fragments.

**Editors & language servers** : error-tolerant incremental trees (this chapter) + name resolution and types on demand (Chapter 12), maintained across edits.

**Exploration** : AST Explorer (astexplorer.net) renders the actual trees of dozens of real parsers side-by-side with source—the fastest way to make everything in this document concrete; Python’s `ast.dump`, Clang’s `-ast-dump`, and Rust’s `cargo expand` are the command-line equivalents.

#### Key idea

Production-grade syntax work adds three guarantees the textbooks omit: *a tree always exists* (error recovery with hole nodes), *the tree remembers everything* (lossless trivia; red–green separation of shareable structure from positional convenience), and *edits cost what they touch* (persistent subtree reuse, incremental reparsing). All three flow from one design commitment made back in §4.5: immutable trees with structural sharing.



## 15 A Builder's Checklist

Everything in this document compresses into a sequence of design decisions. Presented in the order you will face them, each with its governing theory and default recommendation—the document's contents arranged as a working protocol. Even with an AI writing the code, these are the decisions *you* are specifying; they determine the shape of the system more than any line of implementation.

1. **Specify the concrete grammar.** Write the surface syntax as an (E)BNF context-free grammar (Ch. 2). Decide every operator's precedence and associativity and encode the decisions deliberately—by stratified nonterminals, by generator precedence declarations, or by a binding-power table (§2.6, Ch. 7, §8.4); resolve the known ambiguity traps (dangling else, §2.7) on purpose rather than by accident. Push context-sensitive rules out of the grammar and into semantic analysis (Ch. 2, Ch. 12).
2. **Specify the abstract grammar—separately.** Define the AST as its own sum-of-products specification (ASDL-style, §3.3): categories (expr/stmt/decl/type/pattern), constructors, field types. Apply the abstraction test—*drop a token iff no evaluator could need it* (Ch. 3)—and set the fidelity dial (§3.5): conventional AST for a compiler/interpreter; lossless CST-with-abstract-API if the tool will ever write source back out (Ch. 14).
3. **Fix the node payload and representation.** Spans on every node, from day one (§4.2); interned identifiers; an explicit decoration strategy (mutable slots vs. side tables vs. typed rebuild) and mutability policy—immutable-with-sharing if incrementality, concurrency, or tooling are anywhere on the horizon (§4.5, Ch. 14). Choose the encoding for your host: ADT > OO+visitor > homogeneous for safety; flattened/arena if performance is a design goal (§4.3). Plan for `Error`/`Missing` nodes if any editor-adjacent use is conceivable (Ch. 14).
4. **Define the lexer.** Token kinds as regular expressions; maximal munch + priority; values, interning, and spans computed here; trivia policy consistent with the fidelity decision; layout tokens if indentation is significant (Ch. 5).
5. **Choose the parsing method—by the decision procedure of §9.5.** Default for a real front end: hand-written recursive descent for statements (§6.3) with a Pratt/precedence-climbing core for expressions (Ch. 7). Grammar-first projects: an LR generator, left recursion embraced (Ch. 8). Know *why* yours fits: verify the LL(1) conditions if predicting (§6.4), understand your generator's conflicts if reducing (§8.4), respect ordered choice if PEG (§6.5).
6. **Wire tree construction as syntax-directed translation.** S-attributed node-from-children rules wherever possible; inherited accumulation (the loop trick) where left context matters under a top-down method; spans as hulls of child spans (Ch. 10). The grammar the algorithm eats and the tree the language means are different artifacts—the semantic rules are where they reconcile.
7. **Design error handling as a feature.** Earliest-detection messages phrased from the automaton's expectations; panic-mode recovery with chosen synchronization sets; a complete tree



with hole nodes even on broken input if tooling matters (Ch. 14).

8. **Build the passes as folds.** A traversal skeleton (visitor or match-based) with enter/exit moments (Ch. 11); then, in order: name resolution over a scope-stack environment; type checking as inference rules ( $\Gamma \vdash e : \tau$ ) executed as a fold; explicit desugaring into a core; lowering or a tree-walking evaluator as the semantics demand (Chs. 12–13). Prove the delicate ones by structural induction (§11).
9. **Validate against reality.** Round-trip: parse–print–parse should fix-point. Inspect your trees next to production ones (AST Explorer, `python -m ast`, `clang -ast-dump`). Fuzz the parser with malformed input and watch the recovery, not just the acceptance.

The deepest summary this document can offer is a single sentence per part: *a grammar is a blueprint for tree shapes (Ch. 2); an AST is the meaning-preserving quotient of the parse (Ch. 3–4); every parsing algorithm is a strategy for reconstructing the blueprint instantiation behind a string (Ch. 6–9); construction and every later pass alike are algebras scheduled over that tree (Ch. 10–13); and the production-grade extensions all follow from making the tree immutable, lossless, and hole-tolerant (Ch. 14).*

## Further Reading

An annotated guide, ordered roughly as a learning path, mixing the canonical literature with the working resources this document drew on.

### Books

**Robert Nystrom, *Crafting Interpreters* (2021).**

The best single next step, and free online at [craftinginterpreters.com](http://craftinginterpreters.com). Builds a complete language twice—a Java tree-walking interpreter and a C bytecode VM—covering, hands-on, most of this document: recursive descent and Pratt parsing, AST classes and visitors, environments, resolution, and the tree-to-bytecode transition. The chapters *Representing Code* and *Parsing Expressions* are the practical companions to Chapters 3–7 here.

**Aho, Lam, Sethi, Ullman, *Compilers: Principles, Techniques, and Tools*, 2e (2006).**

“The Dragon Book”: the encyclopedic standard reference for the theory—formal grammars, the full LL and LR constructions with proofs, and the definitive treatment of syntax-directed translation and attribute grammars (our Chapter 10 in one hundred rigorous pages). Better consulted than read cover to cover.

**Andrew Appel, *Modern Compiler Implementation in ML/Java/C* (1998).**

The concise modern counterweight to the Dragon Book; famously clean on abstract syntax as algebraic data types, on the visitor pattern, and on the front-to-back architecture of a real compiler. The ML edition is the purest expression of the “AST as ADT, pass as fold” viewpoint of Chapters 4 and 11.

**Grune & Jacobs, *Parsing Techniques: A Practical Guide*, 2e (2008).**

The complete monograph on parsing itself—every algorithm in Chapters 6–9 and dozens more (including the full GLR and Earley stories), with an 1,800-entry annotated bibliography. The reference when you need the *whole* landscape.

**Terence Parr, *Language Implementation Patterns* (2010).**

A patterns-catalog treatment from ANTLR’s author: homogeneous vs. heterogeneous trees, visitors vs. listeners, symbol tables for nested scopes—the engineering middle ground between this document’s theory and production code.

### Foundational papers

**Knuth (1965), “On the translation of languages from left to right.”**

Introduces LR( $k$ ) and proves its equivalence with deterministic pushdown recognition—the founding document of Chapter 8.

**Knuth (1968), “Semantics of context-free languages.”**

Attribute grammars: synthesized/inherited attributes and the circularity question—the foundation of Chapter 10.

**Earley (1970), “An efficient context-free parsing algorithm.”**

Chapter 9’s chart parser, still the reference general algorithm.

**Pratt (1973), “Top down operator precedence.”**

The nud/led design of Chapter 7; strikingly readable. Crockford’s essay *Top Down Operator Precedence* (2007) is the modern JavaScript rendition that revived it.

**Tomita (1985), *Efficient Parsing for Natural Language*.**

GLR and the graph-structured stack.

**Ford (2004), “Parsing expression grammars.”**

Ordered choice, predicates, and (with Ford 2002) packrat linear-time parsing—the basis of §6.5.

**Wadler (1998), “The expression problem.”**

The famous mailing-list statement of §4.4.

## Working resources and real systems

**AST Explorer — [astexplorer.net](http://astexplorer.net).**

Paste code, see the live tree from dozens of production parsers (Babel, TypeScript, Python, and more). The single fastest way to internalize Chapter 3; keep it open while reading.

**Python’s ast module — [docs.python.org/3/library/ast.html](https://docs.python.org/3/library/ast.html).**

A production AST you can hold: `ast.parse`, `ast.dump`, `NodeVisitor`/`NodeTransformer` (the visitor pattern, shipped), and the `Python.asdl` abstract grammar behind it all. *Green Tree Snakes*, the module’s “missing manual,” is the friendly field guide.

**Vaidehi Joshi, “Leveling Up One’s Parsing Game with ASTs” (basecs, 2017).**

The gentlest serious introduction to the CST-vs-AST distinction—good recalibration reading if Chapter 3 ever feels abstract.

**ESTree specification and the Babel plugin handbook.**

The JavaScript ecosystem’s shared homogeneous AST format and the definitive practical guide to writing tree-transforming visitors against it—Chapters 4 and 11 in daily industrial use.

**Roslyn (.NET Compiler Platform) documentation.**

The red-green tree design of Chapter 14 from its source, plus the richest public API for lossless syntax trees. LibCST’s documentation is the Python counterpart.

**tree-sitter — [tree-sitter.github.io](https://tree-sitter.github.io).**

Incremental, error-tolerant GLR parsing behind modern editors; writing a small tree-sitter grammar is an excellent capstone exercise for Chapters 8 and 14.

*A suggested path:* keep AST Explorer open throughout; read Nystrom Part II alongside Chapters 6–7; dip into the Dragon Book or Grune–Jacobs wherever this document says “it can be shown”; and graduate by reading real trees—`python -m ast`, a Babel plugin, a tree-sitter grammar—until the theory here reads as a description of things you have seen.